Digital Signal Processing Exercise Sheet 0

Getting Started: Wokwi and JSLinux

Script: Preface, Chapter 1.1

Learning goals

- Run, modify and share an Arduino project in the Wokwi simulator.
- *Bonus (optional):* compile and run C programs in JSLinux, plot results in the terminal, and transfer measured data from Wokwi (Serial Monitor) to JSLinux.
- Get to know the course library `code/common/engine_signals.h`, the *virtual engine* used on all following sheets.

The virtual engine

Throughout the exercises we cannot measure on a real engine. Instead, a *virtual engine* generates the sensor signals introduced in Chapter 1 of the script. Exactly the same C model is used in three places:

Where	How
JSLinux	`#include "engine_signals.h"` in your C program
Wokwi (sketch)	add `engine_signals.h` as an extra file to the project
Wokwi (custom chip)	part `chip-engine-sim`: outputs CRANK, CAM, KNOCK, ION, and the ground-truth signals TDC and KNK

The conventions of the model are fixed for all sheets:

- 4-cylinder 4-stroke engine, crank angle $\theta \in [0°, 720°)$, firing order 1–3–4–2, combustion TDC at 120°, 300°, 480°, 660°.
- 60-2 trigger wheel: tooth $k = 0, \ldots, 57$ is HIGH for $\theta \bmod 360° \in [6k°, 6k° + 3°)$; teeth 58 and 59 are missing.
- Camshaft signal HIGH for $0° \le \theta < 180°$.
- Knock: damped oscillation with modes at 6.5 kHz and 10.5 kHz, decay time 0.8 ms, onset 5–25° after TDC.
- Disturbances: broadband noise, valve impacts (8 kHz, 90° after TDC), low-frequency combustion vibration, a 1 % speed ripple at twice crankshaft frequency.

Alternative: Wokwi in VS Code

All Wokwi exercises run in the browser; nothing has to be installed. If you prefer working locally, every Wokwi project of the course is also available as a project for the *Wokwi for VS Code* extension in `code/wokwi_vscode/` (same sketches and circuits). There you build the firmware with PlatformIO and Wokwi simulates it; the custom chips come precompiled. You need VS Code with the extensions *PlatformIO IDE* and *Wokwi Simulator* and a free Wokwi license. Setup and workflow are described step by step in `code/wokwi_vscode/README.md`. In the exercises, "open the project in Wokwi" then means: open the folder of the same name in `code/wokwi_vscode/` and edit `src/sketch.ino`. The logic analyzer recording is written to `wokwi.vcd` in the project folder when you stop the simulation.

Exercise 0.1 First steps in Wokwi

WOKWI

Open `https://wokwi.com`, create a new *Arduino Uno* project and replace `sketch.ino` and `diagram.json` by the files in `code/wokwi/sheet00_hello/`.

a) Start the simulation, turn the potentiometer and watch the Serial Monitor. Which ADC value corresponds to 2.5 V? What is the voltage resolution (1 LSB) of the 10-bit ADC?

b) The sketch toggles pin 13 at every sample. Stop the simulation after about one second, download the logic analyzer recording (`.vcd`) and determine the sampling period. Does it match `TS_US`?

c) Change the sampling rate to 1 kHz. Explain why the printed `t_us` values still show regular increments although the `Serial.print` calls take time.

d) Wokwi runs in *simulated time*. Compare the simulated time in the status bar with the time on your wall clock. Why is this important when we later measure execution times?

Exercise 0.2 The engine simulator chip

WOKWI

Every Wokwi project of the following sheets contains the custom chip `engine-sim`. Add the files `engine-sim.chip.c` and `engine-sim.chip.json` from `code/wokwi/chips/` to the Uno project of Exercise 0.1 and add the part `{ "type": "chip-engine-sim", "id": "eng", "attrs": { "vmax": "5", "of` to `diagram.json`. *VS Code:* work in `code/wokwi_vscode/sheet00_hello/`; instead of copying the chip files, add the precompiled chip to `wokwi.toml` (as in `sheet01_crank/wokwi.toml`):

```
[[chip]]
name = 'engine-sim'
binary = '../chips/engine-sim.chip.wasm'
```

a) Connect `eng:CRANK` to logic analyzer D1, `eng:CAM` to D2 and `eng:TDC` to D3 (and `eng:GND` to ground). Record 100 ms at 3000 rpm and view the `.vcd` file (e.g. in PulseView or GTKWave). Identify the missing-tooth gap and verify the tooth period.

b) How many crank teeth lie between the gap and the first TDC pulse? Compare with the conventions above.

c) Change the engine speed with the slider of the chip and repeat the tooth-period measurement at 6000 rpm.

Deliverables

Screenshot of the Wokwi logic analyzer view (or VCD plot) with the gap marked. Bonus (optional): terminal output of `hello_signals`; the CSV-plot program of 0.B1 d).

Bonus: JSLinux (optional)

The lab exercises above use only Wokwi. Interested students can in addition process the engine signals in plain C on a Linux system that runs in the browser. The JSLinux exercises of the following sheets build on this one.

Exercise 0.B1 First steps in JSLinux

JSLINUX

Open `https://bellard.org/jslinux` and start the *x86 Alpine Linux* (console) virtual machine.

a) Check the toolchain: `gcc -version` (or `tcc -v`). Upload `engine_signals.h`, `asciiplot.h`, `csvio.h` (`code/common/`) and `code/bonus/jslinux/sheet00/hello_signals.c` with the upload button below the terminal.

```
gcc -O2 -o hello_signals hello_signals.c -lm
./hello_signals
```

b) Explain the plot of the knock signal: where does the burst start, how long does it last, and what is visible before the burst?

c) Modify the program such that it plots one full engine cycle (720°) at 3000 rpm with `knock_prob = 0`. How many samples are needed at $f_s = 50\,\text{kHz}$? Identify the valve impacts and the combustion vibration in the plot.

d) Copy 2–3 seconds of the Wokwi Serial Monitor output from Exercise 0.1 into a file `pot.csv` in JSLinux (`cat > pot.csv`, paste, `Ctrl-D`). Write a short program that reads it with `csv_read()` and plots the voltage column.

Digital Signal Processing Exercise Sheet 1

The ECU Sees Only Signals

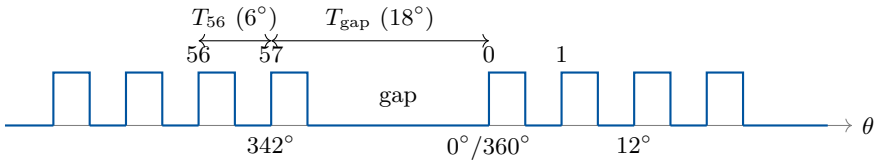
Script: Chapter 1 (The ECU Sees Only Signals)

Learning goals

- Classify the engine sensor signals (continuous/discrete time, analog/binary amplitude, deterministic/stochastic, periodic/aperiodic) and know their characteristic time scales.
- *Bonus (optional):* describe sensor signals quantitatively: mean, RMS, extreme values, autocorrelation as a numerical test for periodicity.
- Extract a first piece of information from a raw signal: crank angle and engine speed from the 60-2 tooth timing, including the effect of the timer resolution.
- Synchronise crank and cam signal to determine the 720° phase of the engine cycle (cylinder identification).

Background: the 60-2 trigger wheel

The crank wheel of the virtual engine has 60 tooth positions (6° each), two of which are missing. The Hall sensor delivers a binary signal; the rising edge of tooth $k = 0, \ldots, 57$ occurs at $\theta = 6k°$ (modulo 360°), the reference mark is the first rising edge after the gap.

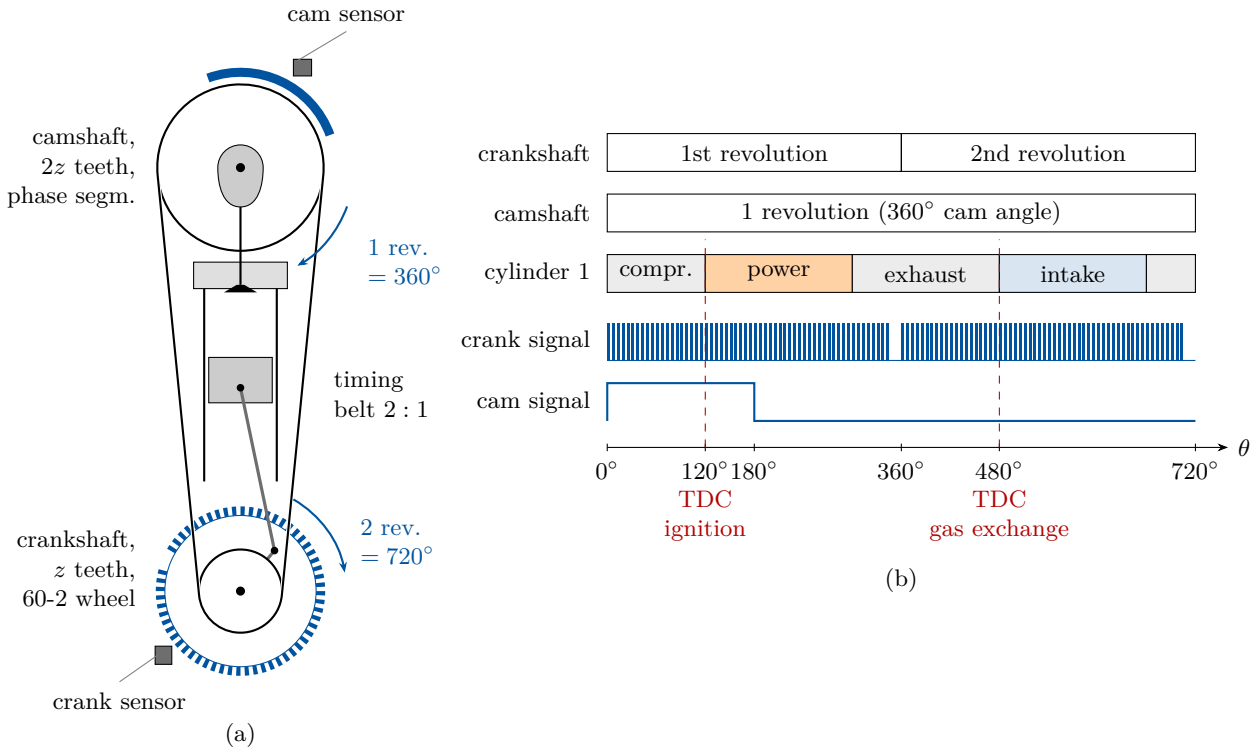


The tooth period T_k is measured from rising edge to rising edge. Because one tooth corresponds to $6° = 1/60$ revolution, the engine speed in rpm is

$$n = \frac{60\,\text{s/min}}{60\,T_k} = \frac{1}{T_k/\text{s}}\ \text{rpm}, \qquad n = \frac{3}{T_{\text{gap}}/\text{s}}\ \text{rpm (gap)}, \qquad n = \frac{60}{T_{\text{rev}}/\text{s}}\ \text{rpm (revolution)}. \tag{1}$$

Background: crankshaft, camshaft and the 720° cycle

A four-stroke engine needs *two* crankshaft revolutions for one working cycle (intake, compression, power, exhaust), but every valve opens only once per cycle. The camshaft is therefore driven by a timing belt (or chain) with a ratio of 2 : 1: its sprocket has twice as many teeth as the crankshaft sprocket, so it turns once while the crankshaft turns twice. The crank angle θ is counted over the full cycle, $0° \le \theta < 720°$, which is exactly one camshaft revolution (360° cam angle). The 60-2 wheel on the crankshaft repeats after 360° crank angle, the single segment of the cam phase wheel only after 720°.



(a) Schematic front view of one cylinder with crank drive and overhead camshaft; the timing belt turns the camshaft once per two crankshaft revolutions. (b) One working cycle of cylinder 1 over the crank angle: the crank wheel pattern and the piston position repeat after 360°, the strokes and the cam signal only after 720°.

Exercise 1.1 Signal classification and time scales

PAPER

a) Classify the five engine signals of Table 1.1 of the script (crank, cam, knock, ion current, lambda) with respect to (i) time: continuous/discrete, (ii) amplitude: analog/binary (digital), (iii) deterministic/stochastic, (iv) periodic/quasi-periodic/aperiodic. Collect the result in a table and justify every entry in one line. Note that the answer may differ for the physical sensor output and for the representation inside the ECU.

b) Discuss the borderline cases: (i) Is the crank signal periodic during an acceleration? Is it periodic in another independent variable? (ii) The knock signal is often called *quasi-periodic*, although a single knock event is a transient. Why? Introduce the notion of a signal whose *statistics* are periodic in the crank angle.

c) Fill in the following table for an engine speed of 3000 rpm: crank revolution period, tooth period and tooth frequency, duration of the gap (rising to rising edge), cam period, cam HIGH time, interval between two combustions (firing frequency), duration of 1° crank angle, time window of a knock onset (5°–25° after TDC), number of 6.5 kHz knock oscillations within one decay time $\tau = 0.8$ ms.

d) Crank and cam: the crankshaft turns 720° per engine cycle, the camshaft 360°. Why is the crank signal alone not sufficient to decide whether the piston of cylinder 1 is in its compression or in its exhaust stroke? After switching on the ignition with the engine cranking at 200 rpm, what is the worst-case time until the ECU knows (i) the crank angle modulo 360°, (ii) the full 720° phase, if the cam level is evaluated at $\theta = 60°$ after the reference mark?

Exercise 1.2 60-2 crank wheel and engine speed

WOKWI

Open the Wokwi project `code/wokwi/sheet01_crank/` (Arduino Uno, engine simulator chip, logic

analyzer, two LEDs, push button). `eng:CRANK` is connected to pin 2 (external interrupt INT0), `eng:CAM` to pin 4. The ISR `crank_isr()` is called at every rising crank edge and time-stamps it. With `TIMEBASE_TIMER1 = 1` the time stamp is the 16-bit Timer1 counter (prescaler 8, 0.5 µs per tick), with 0 it is `micros()` (resolution 4 µs on the Uno).

a) *Preparation (paper).* Derive (1). A timer of resolution Δ quantises the tooth period. Show that the resulting quantisation step of the per-tooth speed is $\Delta n \approx n^2\Delta$ (with n in rpm and Δ in s) and evaluate it for $\Delta = 4\,\mu s$ and $0.5\,\mu s$ at 600, 3000 and 7000 rpm. Compare with the amplitude of the 1 % speed ripple of the model. The gap is detected by the ratio test $T_k > 2\,T_{k-1}$: estimate the angular acceleration (in rpm/s) at which the test would fail (missed gap, or false gap), and compare with realistic engine accelerations ($\lesssim 10\,000$ rpm/s).

b) Complete the ISR: gap detection, tooth counter, per-revolution statistics, TDC LED (pin 12) on for 30° starting at each TDC edge. Which tooth numbers belong to $\theta = 120°$ and 300°? Check with the logic analyzer (D0 = crank, D2 = TDC output of the chip, D3 = your LED) and report the column `teeth` and `sync_err`.

c) Complete the speed computation in `loop()` (per revolution, last tooth, min/max per tooth). At 3000 rpm, compare `rpm_rev` with the spread `rpm_min`…`rpm_max` for both time bases. Press the button to dump 120 tooth periods, copy only this block and plot the per-tooth speed (column `rpm`), e.g. in a spreadsheet or with your CSV plot program of Bonus Exercise 0.B1 d) in JSLinux. Which part of the "jitter" is information, which part is measurement error?

d) Move the speed slider (600…7000 rpm) and repeat c) at 7000 rpm. Why is printing one CSV line per *tooth* not an option (estimate the UART load)?

Exercise 1.3 Cam/crank synchronisation CHALLENGE

Extend your sketch such that it determines the 720° phase from the cam signal (`eng:CAM`, HIGH for $0° \le \theta < 180°$).

a) Why should the cam level not be read at the reference edge ($\theta = 0°$), and which tooth is a good choice instead?

b) Implement `phase` (0° or 360°), toggle it at every gap, and check it at every revolution (`cam_err`). Light the LED on pin 11 only at the compression TDC of cylinder 1 and verify it with the logic analyzer (D1 = cam, D4 = LED).

c) Give the crank angle and the number of the cylinder that fires at each of the four TDC events.

Deliverables

1.1: completed classification table and time-scale table. 1.2: sketch, screenshot of the logic analyzer (gap, TDC, LED), table of rpm statistics for both time bases at 3000 and 7000 rpm, plot of a tooth-period dump. 1.3 (optional): extended sketch, logic analyzer screenshot with cam and cylinder-1 LED. Bonus 1.B1 (optional): program, table of statistics and periods, short interpretation.

Bonus: JSLinux (optional)

For interested students: the following exercise analyses all engine signals offline in JSLinux (set up as in Bonus Exercise 0.B1).

Exercise 1.B1 Signal zoo JSLINUX

The program `code/bonus/jslinux/sheet01/signal_zoo.c` generates 160 ms (four engine cycles) of the crank, cam, knock and ion signals at 3000 rpm, $f_s = 25$ kHz, and 20 s of a lambda sensor signal ($f_s = 200$ Hz) for a slowly oscillating $\lambda(t)$. Additionally, a crank signal during a speed ramp 2000 rpm to 4000 rpm is generated.

a) Complete `stats()`: mean, RMS value, minimum and maximum. Run the program and explain the values of the crank and cam signals analytically. Why is the mean of the knock signal not zero, although the knock sensor is AC coupled?

b) The normalised autocorrelation

$$r[m] = \frac{\hat{R}[m]}{\hat{R}[0]}, \qquad \hat{R}[m] = \frac{1}{N-m} \sum_{n=0}^{N-1-m} (x[n] - \bar{x})(x[n+m] - \bar{x})$$

is close to 1 at lags m where the signal resembles a shifted copy of itself. Complete `autocorr()` and `period_lag()` (first local maximum after the first zero crossing that reaches at least 90 % of the largest value). Report the estimated period and r for all six signals.

c) Interpret the results: Which signals are periodic, which only quasi-periodic? Why does the crank signal show a strong peak already at half a revolution? What happens to the periodicity of the crank signal during the ramp – and which lag is still detected? Why would the $1/N$-normalised (biased) estimate be misleading here?

Digital Signal Processing Exercise Sheet 2

Systems

Script: Chapter 2 (Systems)

Learning goals

- Use the operator view $y = S\{x\}$ and decide for a given system whether it is static or dynamic (memory), causal, linear and time-invariant.
- Understand how a two-point lambda controller with integrator works, why it produces a limit cycle, and predict its period from the loop dead time.
- *Bonus (optional):* design numerical test harnesses that *reveal* a property and know their pitfalls (impulse-only tests, small amplitudes, boundary effects); model engine subsystems as dynamic systems: inertia (first-order lag), wall film, transport delay, switching sensor.

Definitions used on this sheet

For a system $y = S\{x\}$ (continuous or discrete time):

- *memoryless (static)*: $y(t_0)$ depends only on $x(t_0)$;
- *causal*: $y(t_0)$ depends only on $x(t)$, $t \le t_0$; equivalently, if $x_1(t) = x_2(t)$ for $t < t_0$, then $y_1(t) = y_2(t)$ for $t < t_0$;
- *linear*: $S\{ax_1 + bx_2\} = aS\{x_1\} + bS\{x_2\}$ for all inputs and constants (superposition = additivity + homogeneity);
- *time-invariant*: $y(t) = S\{x(t)\} \Rightarrow y(t - t_0) = S\{x(t - t_0)\}$ for every shift t_0 (discrete time: integer shifts n_0).

Linearity and time invariance are previewed here; Chapter 3 shows why their combination (LTI) is so powerful.

Exercise 2.1 Properties by hand

PAPER

a) Decide for each system whether it is memoryless, causal, linear and time-invariant (table with yes/no):

S1: $y(t) = 2x(t)$	S2: $y(t) = x^2(t)$	S3: $y[n] = x[n+1] - x[n]$
S4: $y[n] = \frac{1}{3}(x[n] + x[n-1] + x[n-2])$	S5: $y(t) = x(t - T_d)$, $T_d > 0$	S6: $y(t) = \lvert x(t)\rvert$
S7: $y(t) = \int_{-\infty}^{t} x(\tau)\,\mathrm{d}\tau$	S8: $y[n] = n\,x[n]$	S9: $y[n] = x[n] + 0.5$
S10: $y[n] = x[2n]$	S11: $n_k = 1/T_k$ (Sheet 1)	S12: $U = 0.5 + 0.4\tanh\frac{1-\lambda}{0.005}$

b) Prove your answers formally for S6 (linearity), S8 (time invariance) and S10 (causality and time invariance). For S9, show that the system is not linear, but that the *difference* of two outputs depends linearly on the difference of the inputs (*incrementally linear*).

c) An engineer computes the mean speed of one crank revolution as the arithmetic mean of the 58 per-tooth speeds $1/T_k$, treating the gap period like a normal tooth period. Compute the relative error at constant speed. After correcting the gap ($3/T_{\text{gap}}$), is the arithmetic mean of the per-tooth speeds equal to $60/T_{\text{rev}}$? Which property of S11 is responsible, and what is the error for a speed fluctuation of $\pm 1\,\%$ (sinusoidal)?

Exercise 2.2 Closed lambda loop

WOKWI

Project `code/wokwi/sheet02_lambda/`: Arduino Uno and the custom chip `lambda-probe`. Pin 9 sends the fuel command as PWM (`analogWrite`, 490 Hz, duty $= q/2$), A0 reads the sensor voltage `O2`, A1 the ground truth `LAM` $= \lambda/2$. The chip's sliders set the unmetered air (slider -10 in %, default $+5$ %) and the transport delay (default 200 ms).

How a two-point controller works. The goal is the stoichiometric mixture $\lambda = 1$. The ECU does not know the fuel quantity q^* that achieves it, because part of the air is not measured (leaks, sensor and injector tolerances, ageing): with unmetered air a, $\lambda = (1+a)/q$ and $q^* = 1 + a$. The switching sensor (S12 of Exercise 2.1) delivers only *one bit* of information: $U > 0.45$ V means "rich" ($\lambda < 1$, too much fuel), $U < 0.45$ V means "lean" ($\lambda > 1$, too little fuel) – but not *how* rich or lean. The controller therefore uses the simplest possible rule. In every controller period $T_s = 10$ ms:

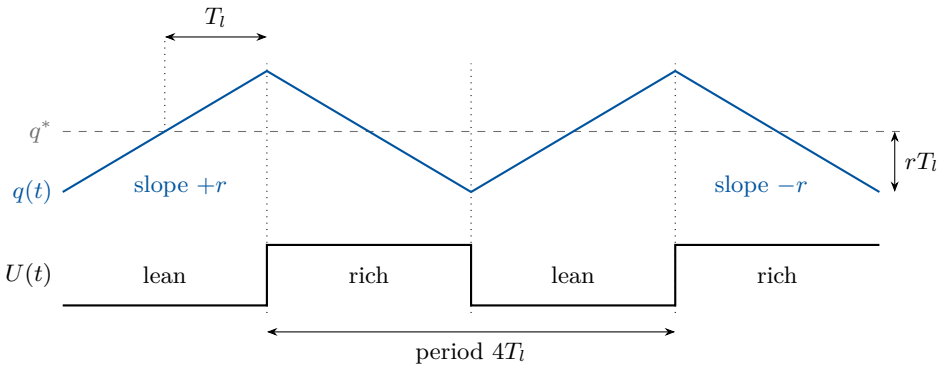
- sensor rich $\Rightarrow$ decrease q by rT_s (ramp down with slope $-r$),
- sensor lean $\Rightarrow$ increase q by rT_s (ramp up with slope $+r$),
- optionally, at the moment the sensor *switches*, an additional jump of q by P towards the other side ($-P$ on lean→rich, $+P$ on rich→lean).

"Two-point" means that the controller output (here the slope $\dot{q}$) takes only two values, $\pm r$, like a relay or a thermostat. "With integrator" means that q is the integral of this output: as long as the sensor reports the same state, q keeps moving in the same direction, so it always moves towards q^*, whatever the value of a. On average the controller therefore finds q^* by itself (integral action).

Why it oscillates. The sensor sees a change of q only after the transport delay T_d and the two lags (wall film $\tau_f = 20$ ms, sensor $\tau_s = 30$ ms), i.e. after an effective dead time $T_l \approx T_d + \tau_f + \tau_s$. When q crosses q^*, the sensor still reports the old state for T_l, and q keeps ramping in the wrong direction: it overshoots by rT_l. After the switching, q needs another T_l to return to q^*, and the sensor sees this crossing only T_l later. Each half period therefore lasts $2T_l$:

$$T_{\text{period}} \approx 4\,T_l, \qquad q = q^* \pm rT_l, \qquad \lambda \approx 1 \mp \frac{rT_l}{q^*}.$$

The oscillation neither grows nor decays – a *limit cycle*. The jump P removes part of the overshoot immediately after each switching, so q reaches q^* earlier (by P/r); the period shortens to $4T_l - 2P/r$.



Limit cycle without jump ($P = 0$): U is the sign of $q - q^*$, delayed by T_l.

a) Implement the two-point controller with integrator ($r = 0.2$/s) in `loop()`. Start the simulation and observe the limit cycle in the Serial Monitor (`u_o2`, `lambda`, `q`). Copy 5 s of data into a spreadsheet (or into JSLinux, see the bonus section) and plot λ and U; compare with Fig. 1.3 of the script and with the sketch above. Predict period and λ range for $T_d = 200$ ms and check them.

b) Measure the oscillation period (column `period_ms`) for transport delays of 100, 200, 400 and 800 ms and plot period over T_d. Determine slope and intercept and interpret them.

c) Move the air slider from +5 % to −10 % and to +10 %. Describe the transient and the new mean value of q.

d) Add the jump `PJUMP` = 0.03, then reduce `RATE` to 0.1 with `PJUMP` = 0.025. Compare period and λ amplitude. Why can no two-point controller make the oscillation disappear, and what would a continuous (PI) lambda controller need?

Deliverables

2.1: property table, proofs. 2.2: sketch, plot of λ and U over 5 s, period over T_d (table + plot), results of c) and d). Bonus 2.B1 (optional): test program, property table with numerical evidence, $h[n]$ of the LTI systems. Bonus 2.B2 (optional): program, values of τ, response times, prediction formula and comparison table.

Bonus: JSLinux (optional)

For interested students: the following exercises identify unknown systems by experiment and simulate the lambda path offline in C. Bonus Exercise 2.B2 c) derives the limit cycle of Exercise 2.2 in detail.

Setup

The following exercises run in JSLinux, a Linux system in the browser. If you have not done Bonus Exercise 0.B1, set it up now (about 10 min):

1. Open `https://bellard.org/jslinux`, start the *x86 Alpine Linux* (console) virtual machine and check the compiler with `gcc -version` (or `tcc -v`).
2. Upload the course headers `engine_signals.h`, `asciiplot.h`, `csvio.h` (`code/common/`) and the files of the exercise (e.g. `code/jslinux/sheet02/`) with the upload button below the terminal; check with `ls` that they arrived. Headers and program must be in the same directory.
3. Compile and run, e.g.

```
gcc -O2 -o blackbox blackbox.c -lm
./blackbox
```

4. The VM forgets all files when the page is reloaded: keep your sources on your own computer and upload them again. Text from Wokwi (Serial Monitor) gets into the VM with `cat > data.csv`, paste, `Ctrl-D`; `export_file data.csv` downloads a file from the VM to your computer.

Exercise 2.B1 Black-box system identification JSLINUX

The header `code/jslinux/sheet02/mystery.h` contains five unknown systems

$$\texttt{void Sk(const double *x, double *y, int N);} \quad k = 1, \ldots, 5$$

(input $x[0..N-1]$, $x[n] = 0$ outside, output $y[0..N-1]$). One of them is LTI; the others violate exactly one or two of the properties of Exercise 2.1. **Do not read the header** – find the properties by experiments, as you would with a supplier's ECU function delivered as object code. The program `code/jslinux/sheet02/blackbox.c` plots the responses to unit impulses at $n_0 = 30$ and $n_0 = 31$ and contains a (weak) homogeneity test with scaled impulses as an example.

a) Superposition test: compare $S\{ax_1+bx_2\}$ with $aS\{x_1\}+bS\{x_2\}$ for random signals and random weights, at amplitude 1 and 0.01. Why is the impulse test of the template not sufficient, and why two amplitudes?

b) Shift test: compare $S\{x[n-k]\}$ with $y[n-k]$ for $k = 1 \ldots 4$. Why are the first and last samples of the test signal set to zero?

c) Causality test: two inputs that agree for $n < n_0$ and differ afterwards.

d) Memory test: two inputs that differ only at n_0. Why must the test be repeated for several n_0?

e) Report a property table with the numerical evidence (largest deviation of each test). For the LTI systems determine $h[n]$ from the plots and give the difference equation. Why does an "impulse response" not characterise the other systems?

Exercise 2.B2 The engine as a dynamic system

JSLINUX

Program: `code/jslinux/sheet02/engine_dyn.c`.

a) *Crankshaft inertia.* With fuel quantity q (1 = nominal), the engine speed obeys $\tau\,\dot{n} = Kq - n$, $\tau = 0.4\,\mathrm{s}$, $K = 2000\,\mathrm{rpm}$. Simulate a fuel step $1 \to 1.5$ at $t = 0.5\,\mathrm{s}$ with the Euler method ($T = 1\,\mathrm{ms}$) and with the exact discretisation $n[k] = a\,n[k-1] + (1-a)Kq[k-1]$, $a = \mathrm{e}^{-T/\tau}$ (derive it!). Determine τ from the 63 % point and compare both methods with the analytic solution at $t = 0.6\,\mathrm{s}$. Is the engine (fuel → speed) static or dynamic?

b) *Lambda path.* Implement `lp_step()` (one step = 1 ms) – it is the model inside the Wokwi chip `lambda-probe`: $q \to$ wall film (first-order lag, 20 ms) $\to \lambda = (1 + a_{\mathrm{air}})/q_f \to$ transport delay T_d $\to$ sensor lag (30 ms) $\to U = 0.5 + 0.4\tanh((1-\lambda)/0.005)$. Classify each block (static/dynamic, linear, TI). Simulate fuel steps $1.00 \to 1.10$ and $1.00 \to 1.20$ ($a_{\mathrm{air}} = +5\,\%$, $T_d = 200\,\mathrm{ms}$) and measure the time until U crosses 0.45 V. Which observations prove that the chain is nonlinear? Under which operating condition is it not time-invariant on a real engine?

c) *Closed loop – offline model of Exercise 2.2.* A two-point controller with integrator changes q with slope $-r$ while the sensor reports rich ($U > 0.45\,\mathrm{V}$) and $+r$ while lean (period 10 ms, 8-bit PWM). Derive the period and the λ amplitude of the resulting oscillation as functions of T_d, r and the lag time constants (hint: a ramp passing a first-order lag τ is delayed by τ). Implement the controller in `closed_loop()`, compare with your prediction for $T_d = 50 \ldots 800\,\mathrm{ms}$, and explain what happens to the mean of q when the unmetered air changes. Extend the prediction to a controller with an additional jump $\pm P$ at every switching of the sensor.

Digital Signal Processing Exercise Sheet 3

LTI Systems

Script: Chapter 3 (Linear Time-Invariant Systems)

Learning goals

- Compute convolutions by hand (graphically and in tables) and in C; verify the algebraic properties of convolution numerically.
- Derive impulse and step responses of the two workhorse smoothers – moving average (FIR) and exponential smoother (IIR) – from their difference equations.
- Quantify the noise-versus-lag trade-off of a smoother (noise gain, mean delay, ramp lag) and relate it to the step response.
- Apply LTI filters to the per-tooth engine speed in real time and understand why the missing-tooth gap must be handled *before* the filter (time invariance in the tooth domain).

Setup: C programs on the simulated ESP32

Exercises 3.2 and 3.3 a) are plain C programs without any circuit: they compute, print tables and draw ASCII plots. They run in Wokwi on an ESP32 that is used as a small C computer – no second environment is needed.

1. Open the project folder `sheet03_conv` (Exercise 3.2) or `sheet03_rpmoffline` (Exercise 3.3 a) in `code/wokwi_vscode/` in VS Code (or open the workspace `sheet03.code-workspace` there, which contains all projects of this sheet). The program is in `src/sketch.ino`; the headers it needs (`asciiplot.h`, `engine_signals.h`) are copies in `src/`.
2. Complete the `TODO`s, build (*PlatformIO: Build*) and start the simulator (`F1` → *Wokwi: Start Simulator*). The program runs once in `setup()` and writes everything to the Serial Monitor; the last line is `done.` Copy the results from there.
3. Why the ESP32 and not the Uno: the programs need tens of kilobytes of RAM (the Uno has 2 KB), and on the Uno `double` is only a 32-bit `float` – the rounding errors of Exercise 3.2 b) would be those of single precision. The ESP32 computes `double` in software: correct, but slow, so Exercise 3.3 a) needs a while in the simulator.
4. The first ESP32 build downloads the toolchain, about 1 GB (see the README in `code/wokwi_vscode/`). In the browser the same projects are in `code/wokwi/`: create an ESP32 project and add the header files as extra tabs.

Background

A discrete-time LTI system is completely described by its impulse response $h[n] = S\{\delta[n]\}$; the output for any input is the convolution

$$y[n] = (x * h)[n] = \sum_{k=-\infty}^{\infty} x[k]\, h[n-k].$$

Two smoothers are used throughout the course (written for the per-tooth speed $r[k]$, but valid for any sequence):

$$\text{MA (FIR):}\quad y[n] = \frac{1}{N}\sum_{k=0}^{N-1} x[n-k] = y[n-1] + \frac{x[n]-x[n-N]}{N},$$

$$\text{EMA (IIR):}\quad y[n] = a\,y[n-1] + (1-a)\,x[n] = y[n-1] + \alpha\,(x[n]-y[n-1]), \qquad \alpha = 1-a,$$

called moving average and exponential smoother (exponential moving average). Useful characteristics of a smoother with $\sum_n h[n] = 1$: the *noise gain* $\sum_n h^2[n]$ (variance reduction for white noise) and the *mean delay* $D = \sum_n n\,h[n]$.

Exercise 3.1 Convolution and smoothers by hand

PAPER

a) Compute $y = x * h$ for $x[n] = \{\underline{1}, 2, 0, -1\}$ and $h[n] = \{\underline{1}, -1, 0.5\}$ (underlined: $n = 0$) twice: graphically (flip and shift h) and with a table of products $x[k]h[n-k]$. Check the result with $\sum_n y[n] = \sum_n x[n] \cdot \sum_n h[n]$ and explain why this identity holds.

b) Determine impulse response $h[n]$ and step response $s[n]$ of the MA of length N and of the EMA (from the difference equation by iteration and from $s[n] = \sum_{k\le n} h[k]$). After how many samples does each reach 50 % and 90 % of a step for $N = 16$ and $a = 15/17$?

c) Compute noise gain and mean delay D of both smoothers. Show that for a ramp input $x[n] = c\,n$ every smoother with $\sum h = 1$ has the steady-state output $y[n] = c\,(n - D)$, and that $D = \sum_{n\ge 0}(1 - s[n])$, i.e. the ramp lag can be read off the step response. Which a gives the EMA the same noise gain as the MA of length N? Compare the mean delays.

d) Show that $x[n] = \mathrm{e}^{\mathrm{j}\Omega n}$ is an eigenfunction of every LTI system, $y[n] = H(\Omega)\,x[n]$ with $H(\Omega) = \sum_k h[k]\mathrm{e}^{-\mathrm{j}\Omega k}$. Evaluate $|H|$ for MA_{16}, MA_{30} and the EMA with $a = 15/17$ at $\Omega = 2\pi/30$. (On the 60-tooth wheel, sampled once per 6°, this is the 2nd engine order – the speed ripple caused by the combustions.)

Exercise 3.2 Convolution engine

WOKWI

Program: project `sheet03_conv` (ESP32 without circuit, see Setup).

a) Implement `conv(x, nx, h, nh, y)` (full linear convolution, output length $n_x + n_h - 1$, no out-of-range accesses) and check it with the example of Exercise 3.1 a).

b) Verify commutativity, associativity and distributivity with random sequences of lengths 200, 50 and 30. Why are the deviations not exactly zero? Why might they be exactly zero for "nice" test numbers?

c) Implement the exponential smoother recursively (`exp_smooth()`) and compare it with the convolution by its impulse response truncated to L samples ($a = 0.9$, $L = 10 \ldots 160$). Relate the error to a^L and compare the number of multiplications per output sample. Verify the step response formula of Exercise 3.1 b).

d) Convolve the impulse response of an 8-point MA with itself. Explain the shape and the mean delay of the result and compare the step responses of one MA and of the cascade.

Exercise 3.3 Smoothing the RPM estimate

WOKWI

The per-tooth speed $r[k] = 1/T_k$ of Sheet 1 contains the real speed fluctuation (2nd order, ±1 %) and timer quantisation. We smooth it with an MA and an EMA in the *tooth domain* (one sample per tooth). Three ways to treat the gap (period $\approx 3T$) are compared: `GAP_MODE` 0 – use $1/T_{\mathrm{gap}}$ like a normal value; 1 – correct it to $3/T_{\mathrm{gap}}$ (one sample); 2 – insert $3/T_{\mathrm{gap}}$ three times (one sample per 6°).

a) *Offline prediction* (ESP32 without circuit, see Setup; project `sheet03_rpmoffline`). The program simulates the crank edges of the virtual engine with 0.5 µs time stamps. Implement the gap handling, `moving_average()` (running sum) and `exp_smoother()`. Report: at constant 3000 rpm mean, standard deviation and min/max of raw, MA_{16}, MA_{30}, $EMA_{2/17}$ for all three gap modes; for a step 3000→5000 rpm the 50 %/90 % times in samples for $N = 8, 16, 32$ and $\alpha = 2/(N+1)$; for a ramp of 3000 rpm/s the lag error in rpm, in samples and in ms. Compare with Exercise 3.1.

b) *Real time* (WOKWI, `code/wokwi/sheet03_rpmfilter/`). The ISR pushes tooth periods into a FIFO; `loop()` computes and filters the speed. Implement the same three steps in the sketch. It prints one line per revolution and, whenever the raw speed jumps by more than 150 rpm between two teeth, a capture block of 128 samples (32 before the jump). Start at 3000 rpm and *click* the speed slider at 5000 rpm. Determine the 50 %/90 % points of both filters from the capture and the ramp lag $D = \sum(1 - s[n])$. (Dragging the slider produces a staircase of steps, not a ramp – why is the step response sufficient anyway?)

c) Repeat with `GAP_MODE` 0 and 1. Explain the per-revolution values and the dips in the capture. Why does the gap violate the time invariance of the tooth-domain processing, and why is only mode 2 compatible with the exact ripple cancellation of MA_{30}?

d) Design: the injection timing needs the speed during an acceleration of 5000 rpm/s at 1000 rpm with an error below 0.5 %, while the 2nd-order ripple should be suppressed. Is this possible with an MA in the tooth domain? What do real ECUs do (keyword: *segment time*)?

Deliverables

3.1: convolution table, derivations of h, s, noise gain, D, $|H|$. 3.2: program and output, explanation of the rounding and truncation errors, plot of the triangular impulse response. 3.3: completed `sheet03_rpmoffline` sketch with result tables; sketch; capture of a step for all three gap modes (plot in any tool, e.g. a spreadsheet), 50/90 % points and ramp lag; answer to d).

Digital Signal Processing Exercise Sheet 4

Sampling

Script: Chapter 4 (Sampling and Quantisation)

Learning goals

- Derive and apply the quantisation-noise model and the rule SNR $\approx 6.02N + 1.76\,\mathrm{dB}$; size an ADC for the knock sensor.
- Configure timer-driven sampling on a microcontroller and verify the sampling rate and the conversion time with the logic analyzer.
- Quantify the effect of sampling-time jitter and compare it with quantisation noise.
- Resample a time-sampled engine signal onto a crank-angle grid using the edges of the 60-2 crank signal, and explain when the time domain and when the angle domain is the natural one.

Setup: JSLinux

From this sheet on, the lab exercises also run in JSLinux, a Linux system in the browser. If you have not done the JSLinux bonus exercises of Sheets 0–2, set it up now (about 10 min):

1. Open `https://bellard.org/jslinux`, start the *x86 Alpine Linux* (console) virtual machine and check the compiler with `gcc -version` (or `tcc -v`).
2. Upload the course headers `engine_signals.h`, `asciiplot.h`, `csvio.h` (`code/common/`) and the files of the exercise (e.g. `code/jslinux/sheet04/`) with the upload button below the terminal; check with `ls` that they arrived. Headers and program must be in the same directory.
3. Compile and run, e.g.

```
gcc -O2 -o quant quant.c -lm
./quant
```

4. The VM forgets all files when the page is reloaded: keep your sources on your own computer and upload them again. Text from Wokwi (Serial Monitor) gets into the VM with `cat > data.csv`, paste, `Ctrl-D`; `export_file data.csv` downloads a file from the VM to your computer.

Background

An ADC with N bits and full-scale range V_{FS} has the step size (1 LSB) $\Delta = V_{\mathrm{FS}}/2^N$. If the signal is "busy" (it moves over many steps between samples), the quantisation error $e = x_q - x$ is well modelled as white noise, uniformly distributed in $[-\Delta/2, \Delta/2)$. Sampling instants that deviate randomly from the ideal grid, $t_n = nT_s + \delta_n$ with $\mathrm{std}(\delta_n) = \sigma_j$ (*jitter*), produce an additional error $e_n \approx \dot{x}(nT_s)\,\delta_n$ that grows with the signal frequency.

An ECU samples two kinds of signals: *time-based* (fixed T_s, e.g. the knock sensor with 20 kHz to 50 kHz) and *angle-based* (one value per fixed crank-angle increment, triggered by the crank wheel). An angle grid can also be computed afterwards from time samples: the crank edges provide pairs (time, angle), between which the angle is interpolated.

Exercise 4.1 Quantisation and jitter budget

PAPER

a) Assume a uniformly distributed quantisation error. Derive its variance and show that a full-scale sine quantised with N bits has SNR $= 6.02N + 1.76\,\mathrm{dB}$. By how much does the SNR drop if the sine uses only 1/10 of the full-scale amplitude?

b) Compute 1 LSB and the rms quantisation noise for the Arduino Uno (10 bit, 0 V to 5 V) and the ESP32 (12 bit, 0 V to 3.3 V).

c) The conditioned knock-sensor signal spans 0 V to 5 V (offset 2.5 V) for the heaviest knock. The weakest knock that still has to be measured has an amplitude of 10 mV; its quantisation SNR must be at least 30 dB. Which dynamic range (in dB) is required, and how many bits does the ADC need?

d) The knock detector only uses a band of width $B = 1\,\text{kHz}$ around the knock frequency, the ADC samples at $f_s = 50\,\text{kHz}$. Assuming white quantisation noise, by how many dB does a band-pass filter improve the SNR of (c)? How many ADC bits does this "save"?

e) Show that jitter limits the SNR of a sine of frequency f to $\text{SNR}_j = -20\log_{10}(2\pi f\sigma_j)$. (i) Which jitter is allowed for a 6.5 kHz knock signal if jitter shall not degrade a 12-bit ADC? (ii) A sampling loop that polls `micros()` (resolution 4 µs on the Uno) produces a timing error uniformly distributed over 4 µs. Compute σ_j, SNR_j at 6.5 kHz and the *effective number of bits* $\text{ENOB} = (\text{SNR} - 1.76)/6.02$.

Exercise 4.2 ADC in practice: timer-driven sampling

WOKWI

Open the Uno project `code/wokwi/sheet04_adc/` (sketch, diagram and the `engine-sim` chip files). Timer1 triggers an interrupt service routine (ISR) at $f_s = 5\,\text{kHz}$ that samples the `KNOCK` output of the chip (A1) into a RAM buffer of 400 samples; the buffer is printed afterwards (the UART cannot transmit 5 kHz × a CSV line in real time). Pin 8 toggles at every sample (logic analyzer D0), pin 9 is HIGH while the ISR runs (D1), the chip's `TDC` output (pin 2, INT0) marks combustion TDCs.

a) Timer1 runs in CTC mode with prescaler 8. Compute the compare value `OCR1A` for 5 kHz and complete `timer1_start()`. What is the lowest sampling rate possible with this prescaler?

b) Complete the ISR. Record about 0.1 s with the logic analyzer and determine from the VCD file (i) the sampling period (D0) and (ii) the time the ISR needs (pulse width on D1). Which sampling rate is the theoretical maximum for this structure? Why is the practical limit lower?

c) Between combustions, 140° to 175° after each TDC, the knock signal contains only sensor noise. Complete the computation of the index range of this quiet window (at 3000 rpm) and run the sketch with the noise slider at 50 mV, 10 mV and 0 mV. Compare the printed standard deviation (in LSB) with your expectation (chip output = 2.5 V + 0.6· sensor signal). Would the 12-bit ESP32 ADC improve the knock measurement here?

d) Turn the potentiometer to an arbitrary position and read the code several times. Is the quantisation error of this constant input "noise"? What does this mean for the validity of the model of Exercise 4.1?

> Wokwi measures time in *simulated* time; the Uno core is simulated instruction by instruction, but treat the measured durations as indicative.

Exercise 4.3 Quantisation noise and jitter

JSLINUX

`code/jslinux/sheet04/quant.c` generates a sine of $f_0 = 1234.5\,\text{Hz}$ at $f_s = 50\,\text{kHz}$ (10^5 samples; f_0/f_s is not a simple fraction, so the error is not periodic). The full-scale range is $[-1, 1)$.

a) Implement the function `quantise()`, a mid-tread quantiser with N bits: rounding to the nearest step and clipping to the codes $q = -2^{N-1}, \ldots, 2^{N-1} - 1$. Implement `snr_db()`. Measure the SNR for $N = 4 \ldots 16$ at full scale and compare with $6.02N + 1.76\,\text{dB}$. For which N does the formula fail, and why?

b) Repeat with an amplitude of 0.1 (−20 dBFS).

c) Implement jittered sampling (Gaussian δ_n, no quantisation) and measure the SNR for $f = 100\,\text{Hz} \ldots 20\,\text{kHz}$ and $\sigma_j = 0.1, 1, 4\,\mu\text{s}$. Compare with $-20\log_{10}(2\pi f \sigma_j)$.

d) Combine a 12-bit quantiser with jitter: compute the ENOB for the knock frequency 6.5 kHz. At which signal frequency are jitter noise and quantisation noise equal for $\sigma_j = 1\,\mu\text{s}$?

Exercise 4.4 Time domain vs. crank-angle domain JSLINUX

In `code/jslinux/sheet04/angle_resample.c` the virtual engine accelerates from 1500 rpm to 4500 rpm within 1 s (`e.rpm` is changed during the simulation). The ion-current signal is sampled at $f_s = 50\,\text{kHz}$; the rising crank edges are time-stamped with a resolution of 2 µs ("input capture" as in a real ECU), and the cam level is latched at every edge.

a) Assign a crank angle to every edge: detect the gap (edge interval $> 2\times$ the previous one), give the first edge after the first gap the angle 0° or 360° (cam level!), then add 6° per edge, 18° across the gap.

b) Resample the signal onto the grid $\theta_m = \theta_0 + m \cdot 0.5°$: for every θ_m interpolate the time $t(\theta_m)$ linearly between the two neighbouring edges, then the signal value $x(t(\theta_m))$ linearly between two time samples. The program compares the grid with the true model angle: report the maximum and rms angle error and explain its size. Rebuild with `-DDT_CAP=20e-6` (edges only known at the sample instants) and compare.

c) The program correlates every engine cycle with the first one, (i) in the angle domain (1440 grid points per cycle) and (ii) in the time domain (segments with the sample count of the first cycle, each starting exactly at its cycle start). Interpret the table and the two plots.

d) The knock-sensor signal contains crank-synchronous parts (combustion vibration $0 \ldots 60°$ after TDC) and fixed-frequency parts (valve impacts: 8 kHz, $\tau = 0.4\,\text{ms}$). How do the two look in the angle domain during the ramp? Compute the angular length of 3τ of a valve burst at 1500 rpm and 4500 rpm.

e) Which angle step $\Delta\theta$ would be needed to sample the 10.5 kHz knock mode directly in the angle domain at 600 rpm and at 6000 rpm? Which signals should an ECU therefore sample angle-based, which time-based?

Deliverables

4.1: derivations and numbers. 4.2: completed sketch, logic-analyzer screenshot with sampling period and ISR duration, quiet-window results for three noise levels. 4.3: SNR tables (quantisation, jitter, ENOB) with short discussion. 4.4: completed program, angle-error values for two capture resolutions, the correlation table and the two plots with your interpretation.

Digital Signal Processing Exercise Sheet 5

Nyquist

Script: Chapter 5 (Sampling Theorem, Aliasing, Reconstruction)

Learning goals

- Predict alias frequencies with the folding rule and the folding diagram; determine minimum sampling rates, including band-pass (under-)sampling.
- Observe aliasing of the knock resonances on real (simulated) microcontrollers and verify the predictions quantitatively.
- Compare zero-order hold, linear and sinc reconstruction.
- Design an anti-aliasing filter: trade-off between filter order, sampling rate and passband loss.

Background

Sampling with $f_s = 1/T_s$ makes all frequencies $f + kf_s$, $k \in \mathbb{Z}$, indistinguishable. A real tone of frequency f therefore appears at the *alias frequency*

$$f_a = |\, f - kf_s|, \qquad k = \mathrm{round}(f/f_s), \qquad 0 \le f_a \le f_s/2.$$

A band-limited signal ($X(f) = 0$ for $|f| \ge B$) is uniquely determined by its samples if $f_s > 2B$ and can be reconstructed by sinc interpolation $x(t) = \sum_n x[n]\,\mathrm{sinc}((t - nT_s)/T_s)$, $\mathrm{sinc}(u) = \sin(\pi u)/(\pi u)$. Everything above $f_s/2$ has to be removed *before* sampling by an analog anti-aliasing (AA) filter.

In our virtual engine the knock sensor has resonances at 6.5 kHz and 10.5 kHz ($\tau = 0.8$ ms) and valve impacts at 8 kHz ($\tau = 0.4$ ms). The `engine-sim` chip updates its analog outputs every 20 µs (attribute `fs_khz = 50`) and holds the value in between: the chip itself is a sampled system with zero-order hold.

Exercise 5.1 Aliasing by hand

PAPER

a) Compute the alias frequencies of the components 6.5 kHz, 8 kHz, 10.5 kHz and of a (hypothetical) third knock mode at 15 kHz for $f_s = 5$, 8.33, 16.67, 25 and 50 kHz. (8.33 kHz is close to the maximum rate of the Uno ADC.)

b) Draw the folding diagram (f_a over f, 0 kHz to 16 kHz) for $f_s = 8.33$ kHz and mark the four components. Which two components can no longer be separated after sampling?

c) What is the minimum sampling rate for a knock signal with components up to 15 kHz? A 4th-order Butterworth AA filter with $f_c = 15$ kHz ($|H| = 1/\sqrt{1 + (f/f_c)^8}$) is used. Which f_s is needed so that every frequency that folds into 0 kHz to 15 kHz is attenuated by at least 40 dB?

d) Band-pass sampling: an analog band-pass passes only the first knock mode, 6 kHz to 7 kHz. Which sampling rates are allowed ($2f_H/k \le f_s \le 2f_L/(k-1)$, $k = 1, 2, \ldots$)? Show that the Uno at 5 kHz could legally capture this band. Where does the band appear, and is it spectrally inverted?

Exercise 5.2 Aliasing made visible

WOKWI

Two projects record the `KNOCK` output of the engine-sim chip (knock probability 100 %, 3000 rpm) together with the ground-truth outputs `KNK` (HIGH during a knock burst) and `TDC`, and print blocks of CSV lines `n,adc,knk,tdc`: `code/wokwi/sheet05_alias_uno/` (Uno, Timer1 ISR, 512 samples per block) and `code/wokwi/sheet05_alias_esp32/` (ESP32, deadline loop, 2500 samples

per block). The JSLinux program `code/jslinux/sheet05/alias_scan.c` cuts out every knock burst (from the rising KNK edge, 4τ) and every valve burst (TDC $+90°$, 5τ), averages the spectra $|X(f)|^2$ of the segments and lists the peaks.

a) Uno: the sampling period is `TS_US = DIV`×20 µs (default `DIV = 10`, 5 kHz). Compute `OCR1A` (prescaler 8) and complete the ISR (store KNK in bit 14, TDC in bit 15). Why does the sketch disable the Timer0 interrupt during acquisition?

b) Copy at least three blocks from the Serial Monitor into `uno.csv`, implement `alias_freq()` and the start index of the valve burst in `alias_scan.c` and run `./alias_scan uno.csv 5000 10`. Compare the spectral peaks with Exercise 5.1. Look at one knock burst in the CSV: how many oscillation periods do you see, and of which frequency? Repeat with `DIV = 6` (8.33 kHz). Check with the logic analyzer that the sample marker (D0) is still regular.

c) ESP32: complete the deadline loop (`TS_US = 40`, 25 kHz) and verify that no sample is late. Analyse with `./alias_scan esp.csv 25000 12`. Then set `TS_US = 60` (16.67 kHz). Why is this particularly dangerous for a knock detector that integrates the energy in a band around 6.5 kHz?

d) Why must the sampling period be a multiple of 20 µs? Consider the Uno at exactly 8 kHz (125 µs): determine the sequence of time offsets between the ADC instants and the last chip update, its effect on the SNR (Exercise 4.1 e) and a value of the chip attribute `fs_khz` in `diagram.json` that fixes the problem.

e) `./alias_scan sim 5000 10` simulates the chip and the ADC ("virtual Wokwi", also writes `alias_sim.csv`). Compare its peaks with your Wokwi measurement.

The UART transmits about 11.5 characters per ms: printing a CSV line per sample is impossible at kHz rates, therefore samples are buffered in RAM (Uno: 512 samples = 1 KB of its 2 KB). Wokwi is not cycle-accurate for the ESP32 – if `late` is not 0, increase `TS_US` to the next multiple of 20.

Exercise 5.3 Reconstruction and anti-aliasing JSLINUX

Reconstruction. `code/jslinux/sheet05/recon.c` represents a "continuous" sine on a grid of $M = 40$ points per sampling interval ($f_s = 25$ kHz, 600 samples) and computes the normalised error $\mathrm{NMSE} = \sum e^2 / \sum x^2$ of a reconstruction on this grid.

a) Implement zero-order hold, linear interpolation and sinc interpolation with $\pm K = 16$ terms, once plain (truncated) and once with a Hann taper $w(u) = \frac{1}{2} + \frac{1}{2}\cos(\pi u/(K+1))$. Show analytically that the ZOH of a sine has $\mathrm{NMSE} = 2(1 - \sin(\omega T_s)/(\omega T_s))$ and compare. Discuss the table (method vs. f/f_s).

b) A 15 kHz tone is sampled at 25 kHz and sinc-reconstructed. What comes out? Predict frequency *and* phase.

Anti-aliasing. `code/jslinux/sheet05/antialias.c` simulates the analog world on a 1 MHz grid; the AA filter (cut-off $f_c = 12$ kHz) is simulated on this grid, the ADC takes every (1 MHz/f_s)-th value. Test signals: knock tones 6.5 kHz/10.5 kHz; a disturbance at $f_d = f_s - 6.5$ kHz that aliases exactly onto the knock frequency; white noise (bandwidth 500 kHz).

c) Implement the RC low-pass (exact update for an input held constant over Δt), the 4th-order Butterworth as a cascade of two 2nd-order sections ($Q_k = 1/(2\sin\frac{(2k-1)\pi}{2n})$, given biquad simulator) and the amplitude measurement at a given frequency in the sampled signal. Run the program for $f_s = 25$ and 50 kHz and filter orders 0, 1, 2, 4. Compare the alias amplitudes with $|H(f_d)|$.

d) Which Butterworth order would be required for 40 dB alias suppression in the band 0 kHz to 10.5 kHz with $f_c = 12$ kHz at $f_s = 25$ kHz and at 50 kHz? Formulate a design recommendation

for the knock ADC.

Deliverables

5.1: alias table, folding diagram, AA and band-pass sampling calculation. 5.2: completed sketches, CSV files of the Uno (two rates) and the ESP32 (two rates), output of `alias_scan` with comparison to 5.1, answers to c) and d). 5.3: completed programs, both tables with discussion, filter-order recommendation.

Digital Signal Processing Exercise Sheet 6

Fourier

Script: Chapter 6 (Fourier Series, Fourier Transform, DTFT)

Learning goals

- Compute Fourier coefficients of engine signals analytically and numerically; understand the line spectrum of the 60-2 crank signal and the effect of the missing teeth.
- Relate the decay time of a resonance to the width of its spectral peak (Lorentzian line shape, bandwidth, quality factor).
- Evaluate the DTFT by direct summation and use its properties (periodicity, symmetry).
- Use *engine orders* to distinguish speed-proportional components from fixed-frequency resonances; track orders during a speed change.

Background

A signal with period P has the Fourier series $x(\theta) = \sum_k c_k\, \mathrm{e}^{\mathrm{j}2\pi k\theta/P}$, $c_k = \frac{1}{P}\int_0^P x(\theta)\mathrm{e}^{-\mathrm{j}2\pi k\theta/P}\,\mathrm{d}\theta$. For crank-angle signals we choose $P = 360°$: then k counts oscillations per crankshaft revolution and is called the *engine order*. At speed n (rpm) order k has the frequency $f = k\,n/60$. Aperiodic signals have a Fourier transform $X(f) = \int x(t)\mathrm{e}^{-\mathrm{j}2\pi ft}\,\mathrm{d}t$; a sampled signal $x[n] = x(nT_s)$ has the DTFT

$$X(\mathrm{e}^{\mathrm{j}\Omega}) = \sum_n x[n]\,\mathrm{e}^{-\mathrm{j}\Omega n}, \qquad \Omega = 2\pi f/f_s,$$

which is 2π-periodic in Ω (f_s-periodic in f) and conjugate symmetric for real $x[n]$. For a finite record of N samples it can be evaluated at any frequency by direct summation; the FFT (Sheet 8) is only a fast way to do this on a regular grid. A sine of amplitude A whose frequency fits an integer number of periods into the record has $|X| = AN/2$, so $A = 2|X|/N$.

Exercise 6.1 Fourier series and transforms of engine signals

PAPER

a) A complete 60-tooth wheel gives a rectangular signal with period 6° and duty cycle 50 %. Compute its Fourier coefficients with respect to $P = 360°$. Which orders occur, what are c_0, $|c_{60}|$, $|c_{120}|$, $|c_{180}|$? Which frequency does order 60 have at 3000 rpm?

b) For the 60-2 wheel (tooth $m = 0, \ldots, 57$ HIGH on $[6m, 6m+3)°$) show that

$$|c_k| = \frac{3}{360}\left|\operatorname{sinc}\frac{k}{120}\right|\left|\frac{\sin(29\pi k/30)}{\sin(\pi k/60)}\right| \quad (k \neq 60m), \qquad |c_{60m}| = \frac{58}{120}\left|\operatorname{sinc}\frac{m}{2}\right|,$$

with $\operatorname{sinc} u = \sin(\pi u)/(\pi u)$. Evaluate c_0, $|c_1|$, $|c_{59}|$, $|c_{60}|$, $|c_{61}|$ and explain the result as a "once-per-revolution modulation". Hint: $x_{60-2} = x_{60} - x_{\mathrm{gap}}$.

c) Compute the Fourier transform of the knock model $x(t) = \mathrm{e}^{-t/\tau}\sin(2\pi f_0 t)$, $t \geq 0$. Show that near f_0 the magnitude is a Lorentzian curve, and derive the -3 dB bandwidth Δf and the quality factor $Q = f_0/\Delta f$. Evaluate for the knock mode ($f_0 = 6.5$ kHz, $\tau = 0.8$ ms) and the valve impact (8 kHz, 0.4 ms). What follows for the bandwidth of a knock band-pass filter?

d) Engine orders: list the frequencies of order 2 (combustions of the 4-cylinder engine), order 60 (crank teeth) and order 0.5 (camshaft) at 1500, 3000 and 6000 rpm, and the order at which the 6.5 kHz knock resonance appears at these speeds.

Exercise 6.2 Numerical Fourier series and DTFT

JSLINUX

a) The program `code/jslinux/sheet06/fourier_series.c` samples one revolution of the crank signal `es_crank()` on a grid of $M = 36000$ points (0.01°). Implement the direct summation

$$c_k \approx \frac{1}{M} \sum_{n=0}^{M-1} x[n]\, \mathrm{e}^{-\mathrm{j}2\pi kn/M}, \qquad k = 0, \ldots, 1200$$

(use a rotating phasor instead of calling `cos` and `sin` for every term) and compare with your results of 6.1 b).

b) Implement the partial sum $S_K(\theta) = c_0 + \sum_{k=1}^{K} 2\,\mathrm{Re}\{c_k \mathrm{e}^{\mathrm{j}2\pi k\theta/360}\}$ and evaluate it around the gap for $K = 60, 180, 600, 1200$. Report maximum, overshoot and rms error. Explain the Gibbs phenomenon: what converges and what does not?

c) `code/jslinux/sheet06/dtft.c`: implement the DTFT by direct summation. For a damped sinusoid ($f_0 = 6.5\,\mathrm{kHz}$, $f_s = 50\,\mathrm{kHz}$, $N = 500$) and $\tau = 0.4/0.8/1.6\,\mathrm{ms}$ the program determines peak frequency, peak value and $-3\,\mathrm{dB}$ bandwidth. Compare with 6.1 c) (note: $X_{\mathrm{DTFT}} \approx f_s X(f)$), and compare the finite sum with the closed-form DTFT of $a^n \sin(\Omega_0 n) u[n]$.

d) Check numerically: $|X|$ at f, $f + f_s$, $-f$ and $f_s - f$. Explain.

e) Apply the DTFT to a real knock burst of the virtual engine (both modes, noise, combustion vibration). Identify all features of the spectrum.

Exercise 6.3 Engine orders

JSLINUX

`code/jslinux/sheet06/orders.c` records the knock-sensor signal of the virtual engine (knock probability 0.2) at constant 1500, 3000 and 6000 rpm, each over exactly 8 engine cycles (16 revolutions, $f_s = 50\,\mathrm{kHz}$), removes the mean and evaluates the amplitude spectrum $A(f) = 2|X(f)|/N$ by direct summation on 0 Hz to 1000 Hz (1 Hz steps) and 4 kHz to 12 kHz (10 Hz steps).

a) Implement `amp_spectrum()`. Why is the record chosen as an integer number of revolutions?

b) The combustion vibration of the model is a half-sine $0.2\,\mathrm{V} \cdot \sin(\pi\varphi/60°)$ for $0 \le \varphi < 60°$ after every TDC (period 180°). Compute its Fourier series and the amplitudes of orders 2, 4, 6, 8; enter your formula in `halfsine_order_amp()`.

c) Run the program. Compare the low-frequency lines (frequency and amplitude) at the three speeds with b). For the high band, the program smooths $|X|^2$ over ±150 Hz and reports the peak in three bands – tabulate frequency and order of the peaks.

d) Explain the fine structure of the high-frequency spectrum: why does the valve resonance appear as a comb of lines with a spacing of order 2 under a Lorentzian envelope, while knock gives a smooth hump?

e) An ECU wants to monitor a crank-synchronous vibration and the knock resonance with fixed band-pass filters. Which of the two filters must be adapted to the engine speed?

Exercise 6.4 Order tracking during a run-up

CHALLENGE

`code/jslinux/sheet06/order_track.c`: the engine accelerates from 1500 rpm to 6000 rpm within 1 s; the knock-sensor signal (no knock) is sampled at 50 kHz, crank edges are time-stamped as in Exercise 4.4.

a) The program computes the amplitude spectrum of the time signal on 0 Hz to 500 Hz. Describe it. Why can no order amplitude be read from it?

b) Resample the signal onto a 0.5° grid (code of 4.4) and compute the "order spectrum" $A(O) = 2|\sum_m x_a[m] \mathrm{e}^{-\mathrm{j}2\pi O m \Delta\theta/360°}|/N_a$ for $O = 0 \ldots 10$ over an integer number of revolutions. Compare the order amplitudes with Exercise 6.3.

c) The angle grid has a "sampling rate" of $360°/\Delta\theta = 720$ samples per revolution. Which orders can be represented, and which signal components alias in the angle domain at 1500 rpm? Why does this not disturb the low orders here?

Deliverables

6.1: derivations, coefficient values, Lorentzian bandwidths, order table. 6.2: completed programs, coefficient table, Gibbs table with explanation, DTFT table and the checks d)/e). 6.3: completed program, order amplitudes (theory vs. measurement) and resonance table with interpretation. 6.4 (optional): order spectrum vs. time spectrum of the run-up.

Digital Signal Processing Exercise Sheet 7

Laplace Transform

Script: Chapter 7 (Laplace transform, transfer functions, stability)

Learning goals

- Describe engine sensors by transfer functions $H(s)$; relate poles and zeros to decay time, resonance frequency, damping, Q and bandwidth.
- Compute impulse and step responses by partial fractions and decide stability from the pole locations.
- Read and sketch Bode diagrams; judge an RC anti-aliasing filter.
- Handle a transport delay e^{-sT_d} by Padé approximation and understand the consequences of a right-half-plane zero.
- Verify analytic results by numerical ODE integration (Euler, RK4) and understand the stability limit of explicit integrators.
- Map an analog system into the z-domain (forward Euler, impulse invariance, bilinear transform) – a preview of Sheets 9 and 10.

Background

For a causal signal $x(t)$ the (unilateral) Laplace transform is $X(s) = \int_0^\infty x(t)\,\mathrm{e}^{-st}\,\mathrm{d}t$. An LTI system described by a linear ODE with constant coefficients has the *transfer function*

$$H(s) = \frac{Y(s)}{X(s)} = \frac{b_0 s^m + \cdots + b_m}{a_0 s^n + \cdots + a_n} = K\,\frac{\prod_k (s - z_k)}{\prod_k (s - p_k)},$$

the Laplace transform of its impulse response $h(t)$. A causal system is BIBO stable iff all poles satisfy $\mathrm{Re}\, p_k < 0$. On the imaginary axis, $s = \mathrm{j}\omega$, $H(\mathrm{j}\omega)$ is the frequency response.

The knock burst of the virtual engine (`code/common/engine_signals.h`) is

$$x_k(t) = A\,\mathrm{e}^{-t/\tau}\sin(2\pi f_d t)\,\varepsilon(t), \qquad f_d = 6.5\,\mathrm{kHz},\ \tau = 0.8\,\mathrm{ms}$$

(plus a second mode at 10.5 kHz). We interpret it as the impulse response of the mechanical path *combustion chamber* → *engine block* → *piezo sensor*, which is excited by the short pressure shock of knocking combustion. Throughout this sheet we use the normalised second-order model ($\sigma = 1/\tau$, $\omega_d = 2\pi f_d$)

$$H_K(s) = \frac{\omega_0^2}{s^2 + 2\sigma s + \omega_0^2}, \qquad \omega_0^2 = \omega_d^2 + \sigma^2, \qquad \zeta = \sigma/\omega_0. \tag{2}$$

Exercise 7.1 Sensor models in the s-domain

PAPER

a) **Knock resonator.** Show that the impulse response of (2) is $h_K(t) = \frac{\omega_0^2}{\omega_d}\mathrm{e}^{-\sigma t}\sin(\omega_d t)\,\varepsilon(t)$. Compute σ, ω_0 (and f_0), ζ, the quality factor $Q = 1/(2\zeta)$, the poles and the region of convergence. Is the system stable? What would $\sigma < 0$ mean?

b) Compute the **step response** of H_K with partial fractions. Give the overshoot, the time of the first maximum and the 2 % settling time. The DC gain is $H_K(0) = 1$: what does this imply for the slow combustion vibration (a half sine over 60° crank angle) that is also present on the sensor signal?

c) **Bode diagram.** Sketch $|H_K(\mathrm{j}\omega)|$ in dB and the phase over $\log f$ (asymptotes!). Compute the height and frequency of the resonance peak and show that the -3 dB bandwidth is $\Delta\omega \approx 2\sigma$, i.e. $B \approx 1/(\pi\tau)$ in Hz. Interpret this relation between decay time and bandwidth.

d) The second knock mode has $f_d = 10.5\,\text{kHz}$ and the same τ. Mark all four poles of the two-mode model in the s-plane. What distinguishes the poles of the two modes, and what do they have in common?

e) **RC anti-aliasing filter.** The knock signal is sampled at $f_s = 40\,\text{kHz}$ behind an RC low-pass with $R = 1\,\text{k}\Omega$, $C = 10\,\text{nF}$. Give $H_{RC}(s)$, its pole and f_c. Compute the attenuation at 6.5 kHz, 10.5 kHz and at 30 kHz. Onto which frequency does a 30 kHz component alias, and why is that critical? Which f_c would a first-order filter need for 40 dB attenuation at 30 kHz? Compare with a second-order Butterworth filter with the same f_c and conclude. Also give the phase and the group delay of the RC filter at 6.5 kHz.

f) **Lambda sensor with transport delay.** Fuel change → sensor is modelled as $G(s) = \mathrm{e}^{-sT_d}/(1 + sT)$ with $T_d = 200\,\text{ms}$ (gas travel) and $T = 30\,\text{ms}$ (sensor), the values of the Wokwi `lambda-probe` chip.

i. Why is $G(s)$ not a rational function, and why is this a problem for the methods of this chapter?

ii. Derive the Padé(1,1) approximation $\mathrm{e}^{-sT_d} \approx (1 - sT_d/2)/(1 + sT_d/2)$ by matching the Taylor series of both sides as far as possible. Where are poles and zeros of the approximated $G_1(s)$?

iii. Compute the step response of $G_1(s)$ with partial fractions. Determine the minimum (time and value) and explain the *undershoot*. Compare with the exact step response.

iv. Phase error: the exact delay has phase $-\omega T_d$, the Padé term $-2\arctan(\omega T_d/2)$. Up to which frequency is the error below 5°? (Use the series $\arctan x \approx x - x^3/3$.)

Exercise 7.2 Knock resonator: ODE solvers vs. Laplace

JSLINUX

Program `code/jslinux/sheet07/resonator_ode.c` integrates the ODE $\ddot{y} + 2\sigma\dot{y} + \omega_0^2 y = \omega_0^2 u$ of (2) in state-space form $x = (y, \dot{y})^T$.

a) Implement `rhs()`, one step of the forward Euler method and one step of the classical Runge–Kutta method (RK4).

b) Implement the analytic impulse and step responses of Exercise 7.1. The program simulates the impulse response (initial state $y(0) = 0$, $\dot{y}(0) = \omega_0^2$ – why?) and the step response for step sizes $h = 40, 20, \ldots, 0.625\,\mu\text{s}$ and prints the maximum error. Estimate the order of convergence p from successive errors ($e(h) \propto h^p$).

c) Explain the results of the Euler method: derive the stability condition $|1 + h\lambda| < 1$ for $\dot{x} = \lambda x$ and evaluate it for the poles of H_K. Compare the limit with the sampling period of the engine chip (20 µs). Why does the chip compute the burst from the closed-form solution instead of integrating an ODE?

d) Implement `freq_resp()` and compare peak, resonance frequency and −3 dB bandwidth with your results of Exercise 7.1 c).

Exercise 7.3 Lambda sensor: delay and Padé approximation

JSLINUX

Program `code/jslinux/sheet07/lambda_pade.c` compares the exact step response of $G(s) = \mathrm{e}^{-sT_d}/(1 + sT)$ with three rational approximations of the delay: a first-order lag $1/(1 + sT_d)$, Padé(1,1) and Padé(2,2)

$$\mathrm{e}^{-sT_d} \approx \frac{1 - sT_d/2 + s^2T_d^2/12}{1 + sT_d/2 + s^2T_d^2/12}.$$

a) Any strictly proper $G(s) = N(s)/D(s)$ can be simulated in *controllable canonical form*: $D(p)\,v = u$ and $y = N(p)\,v$, where $p = \mathrm{d}/\mathrm{d}t$ and state $(v, \dot{v}, \ldots)$. Implement `ccf_rhs()` and `ccf_out()` (integration by RK4 is given).

b) Implement the exact system by a delay line (ring buffer of $D = T_d/h$ samples) followed by the exact discretisation of the lag, $y_{k+1} = a\,y_k + (1-a)\,u_{k-D}$, $a = \mathrm{e}^{-h/T}$, and the analytic step response. Enter the Padé coefficients. Compare minimum, time of $y = 0.5$, RMS and maximum error of the three approximations.

c) Compare the phase of the delay part. Up to which frequency is the phase error of Padé(1,1) and Padé(2,2) below 5°?

d) Which approximation would you use (i) to design a lambda controller, (ii) to predict the sensor signal in the time domain?

Exercise 7.4 From s to z: discretisation of analog systems

JSLINUX

A digital filter shall imitate $H_K(s)$ and $H_{RC}(s)$ at $f_s = 40\,\mathrm{kHz}$, $T = 1/f_s$. Three classical mappings are

$$\text{forward Euler: } s = \frac{z-1}{T}, \qquad \text{bilinear: } s = \frac{2}{T}\,\frac{z-1}{z+1},$$
$$\text{impulse invariance: } h[n] = T\,h(nT) \quad (\text{poles } z_k = \mathrm{e}^{p_k T}).$$

a) **(paper)** Into which region of the z-plane is the left half s-plane mapped by each method, and onto which curve the imaginary axis? Which method can turn a stable analog system into an unstable digital one? Show that the bilinear transform maps ω_a onto $\Omega = 2\arctan(\omega_a T/2)$ (*frequency warping*).

b) Complete the three mapping functions of the program `code/jslinux/sheet07/discretise.c`, i.e. `map_euler`, `map_bilin` and `impinv` (second-order section $H(s) = (b_0 s^2 + b_1 s + b_2)/(a_0 s^2 + a_1 s + a_2)$; impulse invariance via partial fractions with the small complex struct provided).

c) *Prewarping*: with $s = K\frac{z-1}{z+1}$, $K = \omega_r/\tan(\omega_r T/2)$, the frequency ω_r is mapped exactly. Use $\omega_r = \omega_0$ for the resonator and $\omega_r = \omega_c$ for the RC filter.

d) Run the program and interpret the tables: pole radius and angle, peak frequency, gain at DC and close to $f_s/2$. Which method would you choose for the knock resonator, which for the anti-aliasing filter model?

Deliverables

Paper solution of 7.1 including the Bode sketch; the error table and the order estimate of 7.2 with the Euler stability limit; the step-response table of 7.3 and your recommendation 7.3 d); the comparison tables of 7.4 with a short justification of your choice of discretisation method.

Digital Signal Processing Exercise Sheet 8

DFT and FFT

Script: Chapter 8 (discrete Fourier transform, FFT algorithms)

Learning goals

- Compute small DFTs by hand and use the DFT properties (periodicity, symmetry for real signals, Parseval).
- Distinguish frequency resolution $\Delta f = f_s/N$ (record length) from the bin spacing obtained by zero padding.
- Explain leakage and choose a window by main-lobe width, side-lobe level, scalloping loss and equivalent noise bandwidth.
- Implement an iterative radix-2 FFT and verify its $\mathcal{O}(N \log N)$ complexity against the $\mathcal{O}(N^2)$ direct DFT.
- Compute a crank-angle-triggered knock spectrum on the ESP32 and judge the memory and operation budget of an 8-bit Uno.

Background

The N-point DFT of $x[n]$, $n = 0, \ldots, N-1$, and its inverse are

$$X[k] = \sum_{n=0}^{N-1} x[n]\, W_N^{kn}, \qquad x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k]\, W_N^{-kn}, \qquad W_N = \mathrm{e}^{-\mathrm{j}2\pi/N}.$$

$X[k]$ samples the DTFT of the *finite record* at $f_k = k\, f_s/N$. Multiplying by a window $w[n]$ before the DFT convolves the spectrum with the window transform: a sinusoid between two bins leaks into all bins (*leakage*). The radix-2 decimation-in-time FFT splits an N-point DFT into two $N/2$-point DFTs of the even and odd samples,

$$X[k] = G[k] + W_N^k H[k], \qquad X[k + N/2] = G[k] - W_N^k H[k],$$

recursively, giving $\log_2 N$ stages of $N/2$ *butterflies*.

The FFT written on this sheet is used again on Sheets 11 and 12. Its interface is fixed (file `fft.h`):

```
void fft(double *re, double *im, int n, int inverse);
/* in place, n = power of two; inverse = 1 includes the factor 1/n */
```

Exercise 8.1 DFT properties

PAPER

a) Write down the 4×4 DFT matrix and compute the DFT of $x = \{1, 2, 0, -1\}$. Check Parseval's relation $\sum |x[n]|^2 = \frac{1}{N} \sum |X[k]|^2$.

b) Compute the 8-point DFT of $x = \{1, 1, 1, 1, 0, 0, 0, 0\}$ in closed form (geometric series) and give $|X[k]|$. Which symmetry do you observe and why must it hold?

c) **Resolution.** The knock measurement window extends from 10° to 70° after TDC, $f_s = 40\,\mathrm{kHz}$. How many samples does it contain at 3000 rpm and at 6000 rpm, and what is the frequency resolution Δf? Can a knock component at 6.5 kHz be separated from the valve impact at 8 kHz (with a Hann window)? Compare Δf with the natural line width $B = 1/(\pi\tau)$ of the knock burst (Sheet 7).

d) The window of c) at 3000 rpm is zero padded to $N = 256$. What changes in the spectrum, what does not? What *does* improve the resolution?

e) **Windows.** Give main-lobe width (null to null, in bins) and highest side lobe for the rectangular, Hann, Hamming and Blackman windows. The knock component at 6.5 kHz is 60 dB stronger than a valve component at 8 kHz (9.6 bins away for $N = 256$). Which windows can reveal the weak component?

f) **Complexity.** Count complex multiplications of the direct DFT and of the radix-2 FFT for $N = 256$. At 6000 rpm, how many 256-point FFTs per second does an angle-synchronous knock detection for a 4-cylinder engine need, and how many real multiplications per second is that?

Exercise 8.2 Own FFT

JSLINUX

Files: `code/jslinux/sheet08/fft.h` (template of the FFT header)
`code/jslinux/sheet08/fft_test.c` (test program)

a) Implement the direct DFT `dft()` in `fft_test.c`. Avoid N^2 calls of `sin`/`cos`: tabulate W_N^m, $m = 0..N-1$, and use the index $m = (kn) \bmod N$.

b) Implement `fft()` in `fft.h`: bit-reversal permutation, $\log_2 N$ butterfly stages, $1/N$ scaling for the inverse. Compute the twiddle factors of a stage by the recursion $W_L^{k+1} = W_L^k W_L$ (one `sin`/`cos` pair per stage). The program compares FFT and DFT for random complex data, checks $\mathrm{IFFT}(\mathrm{FFT}\, x) = x$, Parseval, and conjugate symmetry. Also check your results of Exercise 8.1 a) and b).

c) Implement `fit_exponent()` (least squares in log–log coordinates). Measure the runtime of DFT and FFT versus N and determine the exponents. Plot $t_{\mathrm{FFT}}/(N \log_2 N)$ – is it constant? Explain deviations for small and for large N.

d) How does the round-off error of the FFT grow with N? Why does the twiddle recursion increase it slightly, and how could one avoid that?

Exercise 8.3 Windows, leakage and the knock spectrum

JSLINUX

Program `code/jslinux/sheet08/windows.c` uses your `fft.h`. Spectra are scaled as amplitude spectra, $A_k = 2|X[k]|/\sum w[n]$, so that a bin-centred sinusoid of amplitude A appears at $20 \log_{10} A$ dB.

a) Implement the periodic Hann, Hamming and Blackman windows. The program computes coherent gain $\sum w/N$, equivalent noise bandwidth $\mathrm{ENBW} = N \sum w^2/(\sum w)^2$, scalloping loss (tone exactly between two bins) and the highest side lobe. Compare with 8.1 e).

b) Leakage: $x[n] = \sin(2\pi\, 6500\, n/f_s) + 10^{-3} \sin(2\pi\, 8000\, n/f_s)$, $N = 256$, $f_s = 40$ kHz. Which windows reveal the weak component? Explain the peak levels with the scalloping loss.

c) Zero padding: two equally strong tones at 6500 Hz and 6900 Hz, records of $N = 64$, 128, 256 samples, each zero padded to 4096 points (Hann). Implement the parabolic interpolation of the peak position $\delta = \frac{1}{2} \frac{A_{k-1} - A_{k+1}}{A_{k-1} - 2A_k + A_{k+1}}$ (dB values). For which record length are the two tones resolved?

d) Knock spectrum: record $N = 256$ samples of `es_knock()` at 40 kHz starting at the TDC of a knocking cylinder (3000 rpm, `knock_prob = 1`), i.e. 0°–115° ATDC, and the window 10°–68° ATDC (128 samples). Identify the peaks. Why is the knock peak higher in the shorter window although it contains less signal energy? Where is the valve impact (8 kHz, 90° ATDC)?

Exercise 8.4 FFT on the microcontroller

WOKWI

Project `code/wokwi/sheet08_knockfft/` (ESP32 + `engine-sim` chip): `KNOCK` → D34 (ADC1), `TDC` → D18, `KNK` (ground truth) → D19. Add your `fft.h` of Exercise 8.2 to the project.

a) The sketch waits for the rising edge of the `TDC` pulse and then records $N = 256$ samples at $f_s = 40\,\text{kHz}$. Which crank-angle range does the record cover at 1500, 3000 and 6000 rpm? Why is the analysis synchronised to the crank angle and not to time?

b) Implement the capture with a deadline loop (`micros()`, period 25 µs); count missed deadlines in `late` and record whether `KNK` was high during the window.

c) Implement the Hann window, the search for the largest bin above 3 kHz and the parabolic peak interpolation. Why must the search exclude the low-frequency range? Set the knock probability to 50 % and collect about 20 windows: compare peak frequency, peak level and the knock-band energy `e_knock` (6–7 kHz) for windows with and without knock (`knk`). Plot one spectrum in JSLinux from the CSV output.

d) **Uno.** Open `code/wokwi/sheet08_unofft/` (synthetic test signal, same `fft.h`; on the AVR `double` is a 32-bit float). Compile for $N = 64$, 128, 256 and read the memory report. What is the largest feasible N and why? Add the operation count (butterflies, real multiplications and additions). Estimate the execution time from the operation count, assuming roughly 100–150 clock cycles per soft-float operation at 16 MHz, and compare with the time between two TDCs at 3000 rpm. Suggest two ways to fit a 256-point spectrum into the Uno.

Hint: Wokwi is not cycle-accurate: `micros()` differences (`tcap`, `tfft`) are simulated times and only indicative. The Serial output of one window (130 lines) takes about 0.25 s at 115200 baud; TDC events during printing are simply skipped.

Deliverables

DFT computations of 8.1; your `fft.h` (it is reused on Sheets 11 and 12!) with the error and runtime tables of 8.2; the window tables and the resolution experiment of 8.3; for 8.4 a CSV spectrum of one knocking and one non-knocking window with your knock/no-knock statistics, and the Uno memory table with the operation-count estimate.

Digital Signal Processing Exercise Sheet 9

Z-Transform

Script: Chapter 9 (z-transform, transfer functions of discrete-time systems)

Learning goals

- Compute z-transforms with their region of convergence (ROC) and invert rational $X(z)$ by partial fractions for causal, anticausal and two-sided sequences.
- Derive $H(z)$, poles and zeros of the RPM smoothers of Sheet 3 and decide stability from the pole positions.
- Read the magnitude response from the pole-zero map (geometric interpretation) and design a two-pole resonator for the knock frequency.
- Build a pole-zero explorer (polynomial roots, ASCII pole-zero map, frequency and impulse response).
- Implement the resonator in Q15 fixed point on the Arduino Uno and observe coefficient quantisation, zero-input limit cycles and overflow oscillations for $r \to 1$.

Background

The (bilateral) z-transform of $x[n]$ is $X(z) = \sum_{n=-\infty}^{\infty} x[n] z^{-n}$, defined in a ring-shaped ROC $r_1 < |z| < r_2$. An LTI system with the difference equation $\sum_k a_k y[n-k] = \sum_k b_k x[n-k]$ has the transfer function

$$H(z) = \frac{b_0 + b_1 z^{-1} + \cdots + b_M z^{-M}}{a_0 + a_1 z^{-1} + \cdots + a_N z^{-N}} = \frac{b_0}{a_0}\, z^{N-M}\, \frac{\prod_k (z - z_k)}{\prod_k (z - p_k)}.$$

A causal system is stable iff all poles lie inside the unit circle (then the ROC $|z| > \max |p_k|$ contains the unit circle), and its frequency response is $H(\mathrm{e}^{\mathrm{j}\Omega})$, $\Omega = 2\pi f / f_s$. Geometrically, $|H(\mathrm{e}^{\mathrm{j}\Omega})| = |\frac{b_0}{a_0}| \prod_k |\mathrm{e}^{\mathrm{j}\Omega} - z_k| / \prod_k |\mathrm{e}^{\mathrm{j}\Omega} - p_k|$: a pole close to the unit circle produces a peak, a zero on it a notch.

The **two-pole resonator** with poles $p_{1,2} = r\mathrm{e}^{\pm \mathrm{j}\theta}$, $\theta = 2\pi f_0 / f_s$,

$$H(z) = \frac{b_0}{1 + a_1 z^{-1} + a_2 z^{-2}}, \qquad a_1 = -2r\cos\theta, \quad a_2 = r^2, \tag{3}$$

is the simplest digital knock filter. Throughout this sheet $f_0 = 6.5\,\mathrm{kHz}$ and $f_s = 40\,\mathrm{kHz}$ ($\theta = 58.5°$).

Exercise 9.1 Analytical z-transforms

PAPER

a) Compute the z-transform and the ROC of $\delta[n-k]$, $\varepsilon[n]$, $a^n\varepsilon[n]$, $-a^n\varepsilon[-n-1]$ and $r^n \cos(\theta n)\,\varepsilon[n]$. Why is the ROC indispensable?

b) Find all sequences with $X(z) = \dfrac{1}{(1 - 0.5z^{-1})(1 - 0.8z^{-1})}$ by partial fractions (one for each possible ROC). Which one is causal, which one has a DTFT? Check $x[0]$ and $x[1]$ of the causal one with the difference equation.

c) **RPM smoothers of Sheet 3.** Derive $H(z)$, poles and zeros of

   i. the exponential smoother $y[n] = a\,y[n-1] + (1-a)\,x[n]$,

   ii. the M-point moving average.

   When are they stable? Give $|H|$ at $\Omega = 0$ and $\Omega = \pi$ for $a = 0.9$ and the notch frequencies of the moving average for $M = 8$ at $f_s = 40\,\mathrm{kHz}$. What happens to the pole at $z = 1$ of $(1 - z^{-M})/(1 - z^{-1})$?

d) **Resonator.** Show that (3) has the poles $re^{\pm j\theta}$. Derive the stability triangle in the (a_1, a_2)-plane ($|a_2| < 1$, $|a_1| < 1 + a_2$) and mark the resonator.

e) Using the geometric interpretation, show that the $-3\,\mathrm{dB}$ bandwidth is $\Delta\Omega \approx 2(1-r)$, i.e. $B \approx (1-r)f_s/\pi$, for r close to 1. Choose r for $B = 400\,\mathrm{Hz}$. Compare with the impulse-invariant image $r = e^{-T/\tau}$ of the analog knock resonator of Sheet 7 ($\tau = 0.8\,\mathrm{ms}$).

f) Determine b_0 such that $|H(e^{j\theta})| = 1$ and evaluate it for $r = 0.97$.

Exercise 9.2 Pole-zero explorer

JSLINUX

Program `code/jslinux/sheet09/pz_explorer.c` reads the coefficients from the command line,

```
./pz_explorer -fs 40000 -b 0.1 -a 1 -0.9      # exponential smoother
./pz_explorer -fs 40000 -res 6500 0.97        # resonator, gain 1 at f0
```

computes poles and zeros, prints them in polar form (radius, angle in Hz), decides stability, draws the pole-zero map with `ap_pz()` and plots magnitude (dB) and phase over f and the impulse response.

a) Poles and zeros are the roots of $z^L B(z)$ and $z^L A(z)$, $L = \max(M, N)$ (`tf_roots()` is given). Implement the Durand–Kerner iteration `poly_roots()` for a polynomial of degree n with $c_0 = 1$:

$$r_i \leftarrow r_i - \frac{c(r_i)}{\prod_{j \neq i}(r_i - r_j)}, \qquad r_i^{(0)} = (0.4 + 0.9j)^{i+1}.$$

Use the small complex struct of the template (no `complex.h`).

b) Implement the geometric evaluation `freq_geom()` and the impulse response from the difference equation. The program compares the geometric magnitude with the direct evaluation of B/A.

c) Explore and report (pole-zero map, peak/notch, bandwidth, $|h[n]|$):

i. exponential smoother $a = 0.9$; moving average $M = 8$;

ii. resonator $r = 0.9$, 0.97, 0.99: compare the $-3\,\mathrm{dB}$ bandwidth with $(1-r)f_s/\pi$;

iii. resonator $r = 1$ and $r = 1.02$: what does the program say, what does the impulse response do?

iv. a notch filter at 6.5 kHz with zeros at $e^{\pm j\theta}$ and poles at $0.95e^{\pm j\theta}$. Give the coefficients and the notch width; why is $|H| \approx 1$ far from the notch?

d) Why may the magnitude plot of an unstable system be misleading?

Exercise 9.3 Digital resonator in float and Q15

WOKWI JSLINUX

The resonator (3) is implemented twice in the header `resonator.h`: in `float` and in fixed point with signals in Q15 (`int16_t`, $1\,\mathrm{LSB} = 2^{-15}$), coefficients in Q14 and an `int32_t` accumulator. The *same file* is used on JSLinux and on the Uno; you find it in both folders `code/jslinux/sheet09/` and `code/wokwi/sheet09_resonator/`.

a) **(paper)** Why can a_1 not be stored in Q15 here? In which format is the accumulator, and how many bits does it need at most? Which quantisation step does $a_2 = r^2$ get, and how large is the resulting change of the pole radius for $r = 0.9999$?

b) Implement the functions `res_design`, `res_f_step` and `res_q_step` (rounding or truncation, saturation or wrap-around, and count the overflows). Note: `int` has 16 bits on the AVR – cast to `int32_t` *before* multiplying. Run the program `limitcycle.c` and compare quantised and exact pole positions (part (a) of the program) and the impulse responses (b).

c) **Zero-input limit cycles.** Starting from $y[-1] = 0.5$ with $x = 0$, the float resonator decays to zero. Part (c) of the program measures the amplitude of the Q15 output after 5 s. For rounding, a limit cycle can persist as long as rounding maps $a_2 y$ back to y: derive the dead-band bound $|y| \leq \frac{0.5}{1-a_2}$ LSB. Compare with the measured amplitudes for $r \to 1$, rounding and truncation, and explain the frequency of the limit cycle.

d) **Overflow.** Without normalisation ($b_0 = 1$) the gain at f_0 is about 20 (Exercise 9.1 f). Part (d) feeds a knock burst of amplitude $A = 0.1$ and 0.5 into the resonator with $r = 0.99$. Compare wrap-around and saturation. What is an *overflow oscillation*?

e) **Uno.** Open the Wokwi project (Uno, potentiometer = r from 0.90 to 1.00, switch SW1 = rounding/truncation, SW2 = saturation/wrap, button = next test T0/T1/T2, see the sketch header) with your `resonator.h`. Reproduce the results of c) and d) for $r \approx 0.97$, 0.99, 0.999. Are the integer results identical to JSLinux? Could you also expect identical float results?

f) Draw conclusions for a fixed-point knock filter in an ECU (choice of r, rounding, overflow handling, scaling).

Hint: The Uno computes the test signals itself; its ADC (about 9 kHz maximum, Sheet 5) could not sample a 6.5 kHz knock signal anyway. The tests are not real-time – we study the arithmetic; real-time knock filtering follows on Sheet 10.

Deliverables

Solutions of 9.1 (partial fractions with all ROCs, stability triangle, bandwidth formula and r for 400 Hz); the pole-zero explorer with the screenshots (ASCII plots) and the result table of 9.2 c); the tables of 9.3 a), c) and d) from JSLinux and the corresponding Uno outputs (T0–T2 for at least two values of r) with your conclusions 9.3 f).

Digital Signal Processing Exercise Sheet 10

Digital Filters

Script: Chapter 10 (FIR and IIR filter design, filter structures)

Learning goals

- Turn a verbal requirement ("extract the knock resonance") into a quantitative filter specification and estimate the necessary order.
- Design linear-phase FIR filters with the window method and IIR filters with the bilinear transform (including pre-warping); realise IIR filters as biquad cascades.
- Judge the effects of finite word length: coefficient quantisation, scaling, overflow, saturation; Q15 arithmetic on an 8-bit controller.
- Run the knock band-pass sample by sample in real time on the ESP32 (float) and on the Arduino Uno (fixed point) and compare FIR and IIR in terms of cost and delay.

Background and conventions

The knock sensor signal of the virtual engine contains the knock resonance at $f_1 = 6.5\,\text{kHz}$ (plus a weaker mode at 10.5 kHz), valve impacts at 8 kHz, a low-frequency combustion vibration and broadband noise. The first block of every knock detector is a band-pass filter around f_1. On this sheet we use

sampling rate	$f_s = 25\,\text{kHz}$ ($T_s = 40\,\mu\text{s}$)
pass band	6.0 kHz to 7.0 kHz, loss at most $A_p = 1\,\text{dB}$
stop bands	$f \le 5.0\,\text{kHz}$ and $f \ge 8.0\,\text{kHz}$, attenuation at least $A_s = 40\,\text{dB}$

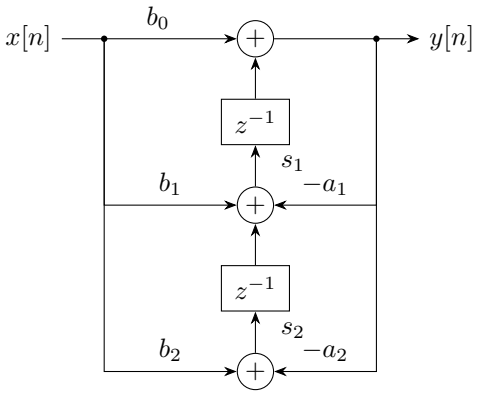
A biquad (second-order section) is written

$$H_k(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}},$$

a cascade is $H(z) = \prod_k H_k(z)$. Two structures are used: *direct form I* (DF I, states $x[n-1], x[n-2], y[n-1], y[n-2]$, one accumulator – preferred in fixed point) and the *transposed direct form II* (TDF II, two states s_1, s_2 – preferred in floating point):

$$y = b_0 x + s_1, \qquad s_1 \leftarrow b_1 x - a_1 y + s_2, \qquad s_2 \leftarrow b_2 x - a_2 y.$$

Q15 denotes a 16-bit two's complement number $v = q/2^{15} \in [-1, 1)$, *Q2.14* a 16-bit number $v = q/2^{14} \in [-2, 2)$.



Biquad in transposed direct form II.

Exercise 10.1 Design by hand

PAPER

a) *Sampling rate.* The engine-sim chip updates its analog outputs every 20 µs (50 kHz, zero-order hold). Explain why the ESP32 samples the knock signal at $f_s = 25\,\text{kHz}$ and not, e.g., at 40 kHz. Estimate the worst-case phase error of the 6.5 kHz component caused by the timing mismatch at 40 kHz. Is 25 kHz sufficient for both knock modes and for the valve disturbance? What happens to the white noise of the chip (generated at 50 kHz) when only every second value is sampled?

b) *FIR order.* Normalise the specification ($\Delta F = \Delta f / f_s$). Estimate the length N of a window-design FIR filter for a Hamming window ($N \approx 3.3/\Delta F$) and with Kaiser's formula $N - 1 \approx (A_s - 8)/(2.285\,\Delta\omega)$, $\beta = 0.5842(A_s - 21)^{0.4} + 0.07886(A_s - 21)$. Where do you put the cut-off frequencies of the ideal band-pass?

c) Derive the impulse response $h_d[m]$ of the ideal (zero-phase) band-pass with cut-offs $\omega_{c1} < \omega_{c2}$, and state how the causal FIR filter $h[n]$, $n = 0, \ldots, N-1$, is obtained from it. What is its group delay in samples, in ms, and in degrees crank angle at 3000 rpm and 6000 rpm? Why does this matter for a knock measurement window of 10°–70° after TDC?

d) *IIR order.* Pre-warp the band edges, $\Omega = 2 f_s \tan(\pi f / f_s)$, and compute the stop-band frequency $\lambda_s = \min |\Omega_s^2 - \Omega_0^2| / (B\,\Omega_s)$ of the normalised low-pass prototype ($\Omega_0^2 = \Omega_{p1}\Omega_{p2}$, $B = \Omega_{p2} - \Omega_{p1}$). Which Butterworth prototype order n is needed, how many biquads does the band-pass have, and how many multiplications per sample does it cost compared with the FIR filter?

e) *Second-order band-pass.* The knock burst decays with $\tau = 0.8\,\text{ms}$. Show that its spectrum has a −3 dB bandwidth of $B_k = 1/(\pi\tau)$ and argue why a band-pass with about this bandwidth is a good compromise (matched filter). Design $H(s) = Bs/(s^2 + Bs + \Omega_0^2)$ with centre $f_0 = 6.5\,\text{kHz}$ and digital bandwidth 400 Hz via the bilinear transform $s = 2f_s\,(1 - z^{-1})/(1 + z^{-1})$ with pre-warping (use $B \approx 2\pi \cdot 400\,\text{Hz}/\cos^2(\pi f_0 / f_s)$). Compute b_0, b_1, b_2, a_1, a_2, the pole radius r and the decay time of the filter's impulse response.

Exercise 10.2 Filter design tool

JSLINUX

Files in `code/jslinux/sheet10/`: `filtdesign.h` (design library), `filterdesign.c` (design, check, export), `coefquant.c` (word-length study), `filtertest.c` (test on the virtual engine).

```
gcc -O2 -o filterdesign filterdesign.c -lm && ./filterdesign    # writes knock_filter.h
gcc -O2 -o coefquant coefquant.c -lm && ./coefquant && ./coefquant 200000
gcc -O2 -o filtertest filtertest.c -lm && ./filtertest
```

a) *FIR window method.* Complete `fd_window()` (Hann, Hamming, Blackman, Kaiser) and `fd_fir_bandpass()`. `filterdesign` prints, for $N = 83$ and every window, the worst pass-band loss and stop-band gain, and the smallest odd N that meets the specification. Explain the ranking of the windows. Is the Kaiser estimate of 10.1 b) met?

b) *IIR Butterworth band-pass.* Complete `fd_prewarp()` and the pole transformation in `fd_butter_bandpass()`: every prototype pole p gives the band-pass poles $s = \frac{1}{2}(pB \pm \sqrt{p^2B^2 - 4\Omega_0^2})$, which are mapped with $z = (2f_s + s)/(2f_s - s)$. Check the specification, list the pole radii and frequencies of the sections, and compare the group delay at 6.5 kHz with the FIR filter. What goes wrong without pre-warping?

c) Complete `fd_bp2()` and compare with your hand calculation of 10.1 e). Report the attenuation of the 400-Hz and a 1000-Hz version at 8 kHz and 10.5 kHz.

d) *Export.* Complete the Q15/Q2.14 conversion (rounding, saturation) and generate `knock_filter.h`. Why are the biquad coefficients exported in Q2.14 rather than in Q15? Why are the sections scaled to a peak gain of 1 (L_∞ scaling) and ordered by pole radius?

e) *Coefficient quantisation.* Complete `quant_vec()` and the quantisation of a_1, a_2 in `coefquant.c`. The program realises the filter (1) as a biquad cascade and (2) as one 10th-order direct-form polynomial, quantises the coefficients to $B = 8 \ldots 32$ bits and reports the pole displacement, the largest pole radius, the pass-band deviation and the stop-band gain. Interpret the results for $f_s = 25\,\text{kHz}$ and for the same analog specification at $f_s = 200\,\text{kHz}$. Which word length do you need for the cascade, which for the direct form?

f) *Test on the engine signal.* Complete `sos_step()` (TDF II, float) and `sosq_step()` (DF I, Q15 data, Q2.14 coefficients, 32-bit accumulator, rounding, saturation) in `filtertest.c`. Compare the window energies with and without knock for FIR and IIR, the delay of the filtered burst, and the error of the Q15 implementation.

Exercise 10.3 Real-time filtering on the microcontroller

WOKWI

a) *ESP32, float.* Open `code/wokwi/sheet10_esp32_filter/` and replace the placeholder `knock_filter.h` by your generated header. The sketch samples the KNOCK pin at 25 kHz (deadline loop) and records 300 samples after each TDC of cylinder 1 (TDC pulse while CAM is HIGH). Complete `fir_step()` (circular buffer stored twice, so the newest N samples are always contiguous) and `iir_step()` (TDF II).

i. Run at 3000 rpm with knock probability 100 % and 0 %. Copy one record into JSLinux and plot x, y_{FIR}, y_{IIR} and the KNK flag. Compare the window energies printed in the `# rec` lines.

ii. Repeat at 6000 rpm. Explain the drastic change of the FIR window energy and propose a fix.

iii. Why does the sketch use `float` and not `double` on the ESP32? Compare the indicative benchmark of FIR and IIR with the operation counts.

b) *Uno, Q15.* Open `code/wokwi/sheet10_uno_q15/`. Timer1 interrupts at 25 kHz; the ISR reads the ADC, starts the next conversion and filters with the single biquad `kf_bp2_q14`.

i. Complete `biquad_q15()`: DF I, 32-bit accumulator (Q29), rounding, conversion to Q15, overflow detection, saturation or wrap-around (`SATURATE`). Verify it in `MODE 0`, where a recorded block is filtered with float and with Q15 arithmetic.

ii. Cycle budget: how many CPU cycles are available per sample? Measure the ISR duration (column `isr_cycles`, Timer1 count at the end of the ISR, or the pulse width on D8 with the logic analyzer). How many biquads, how many FIR taps would fit? Could the 10th-order cascade of 10.2 run on the Uno at 25 kHz?

iii. Scaling: the input is mapped by $x = (\text{adc} - 512) \cdot 2^{\text{SHIFT}}$. Which `SHIFT` uses the full Q15 range? Show that the band-pass output can still overflow when the *input* clips (hint: a clipped sine approaches a square wave). Run `MODE 1` with knock amplitude 3 V and `SHIFT` = 6, 8, 10 and compare saturation with wrap-around.

iv. Complete the window energy `energy += y*y >> 8`. Show that the 32-bit accumulator cannot overflow down to 600 rpm. Why is $y^2 \gg 8$ much cheaper on the AVR than $y^2 \gg 12$?

Deliverables

10.1 on paper. `knock_filter.h` and the output of `filterdesign` and `coefquant` (both sampling rates); plot of one ESP32 record (x, y_{FIR}, y_{IIR}, KNK) at 3000 and 6000 rpm; Uno: block-test error, measured ISR cycles, table of overflow events for the three shifts with a short discussion.

Digital Signal Processing Exercise Sheet 11

Maximum Entropy Spectral Estimation

Script: Chapter 11 (AR models, Yule–Walker, Levinson–Durbin, Burg)

Learning goals

- Describe a resonance by an autoregressive (AR) model and relate the AR poles to resonance frequency and damping.
- Derive and solve the Yule–Walker equations with the Levinson–Durbin recursion; implement Burg's method; explain the maximum entropy principle.
- Compare MEM (AR) spectra with windowed periodograms on the very short records available in a knock window (32–128 samples): frequency accuracy, peak shape, resolution, artefacts.
- Select the model order (FPE, AIC, MDL) and recognise the symptoms of a wrong order (spurious peaks, line splitting).

Background

A knock window is short: 60° crank angle are 3.3 ms at 3000 rpm and only 1.7 ms at 6000 rpm, i.e. 42–83 samples at $f_s = 25\,\text{kHz}$ (the rate of Sheet 10). The DFT of N samples has the bin spacing f_s/N (781 Hz for $N = 32$), and windowing widens the peaks further. Parametric methods fit a model to the data and are not bound to this "Rayleigh limit".

An AR(p) process is white noise $e[n]$ (variance σ^2) filtered by an all-pole filter:

$$x[n] = -\sum_{i=1}^{p} a_i\, x[n-i] + e[n], \qquad A(z) = 1 + \sum_{i=1}^{p} a_i z^{-i}, \qquad P(f) = \frac{\sigma^2\, T_s}{\left|A(\mathrm{e}^{\mathrm{j}2\pi f/f_s})\right|^2}.$$

Multiplying by $x[n-m]$ and taking expectations gives the *Yule–Walker equations* $\sum_{i=1}^{p} a_i\, r[|m-i|] = -r[m]$, $m = 1, \ldots, p$, and $\sigma^2 = r[0] + \sum_i a_i r[i]$. The *Levinson–Durbin recursion* solves them order by order in $O(p^2)$ operations ($E_0 = r[0]$):

$$k_m = -\frac{r[m] + \sum_{i=1}^{m-1} a_i^{(m-1)} r[m-i]}{E_{m-1}}, \quad a_i^{(m)} = a_i^{(m-1)} + k_m\, a_{m-i}^{(m-1)}, \quad a_m^{(m)} = k_m, \quad E_m = (1-k_m^2)E_{m-1}.$$

Burg's method uses the same order recursion, but estimates every reflection coefficient k_m directly from the data by minimising the sum of forward and backward prediction error powers,

$$k_m = -\frac{2\sum_{n=m}^{N-1} f_{m-1}[n]\, b_{m-1}[n-1]}{\sum_{n=m}^{N-1}\left(f_{m-1}^2[n] + b_{m-1}^2[n-1]\right)}, \qquad \begin{aligned} f_m[n] &= f_{m-1}[n] + k_m b_{m-1}[n-1], \\ b_m[n] &= b_{m-1}[n-1] + k_m f_{m-1}[n], \end{aligned}$$

with $f_0[n] = b_0[n] = x[n]$. No autocorrelation estimate (and hence no implicit windowing of the data) is needed. Order selection criteria (E_p = prediction error power of order p, N = record length):

$$\mathrm{FPE}(p) = E_p \frac{N+p+1}{N-p-1}, \qquad \mathrm{AIC}(p) = N \ln E_p + 2p, \qquad \mathrm{MDL}(p) = N \ln E_p + p \ln N.$$

Code: `code/jslinux/sheet11/`: `ar.h` (AR library), `fft.h` (radix-2 FFT with the API of Sheet 8, `fft(re, im, n, inverse)` – you may use your own), `memspec.c` (Exercise 11.2), `resolve.c` (Exercise 11.3).

Exercise 11.1 AR models and the maximum entropy principle

PAPER

a) AR(1): derive a_1 and σ^2 from $r[0], r[1]$ and the PSD. For which sign of a_1 is the process low-pass, for which high-pass?

b) AR(2): write down the Yule–Walker equations and solve them for a_1, a_2 in terms of $\rho_1 = r[1]/r[0]$, $\rho_2 = r[2]/r[0]$. Then run the Levinson–Durbin recursion by hand for the normalised autocorrelation $r = [1,\ -0.0627,\ -0.8973]$ and give $k_1, k_2, a_1, a_2, E_1, E_2$.

c) Show that an AR(2) model with complex poles $z_{1,2} = re^{\pm j\theta}$ has $a_1 = -2r\cos\theta$, $a_2 = r^2$ and describes a resonance. Compute r, θ, a_1, a_2 for the knock mode ($f_1 = 6.5\,\text{kHz}$, decay time $\tau = 0.8\,\text{ms}$, $f_s = 25\,\text{kHz}$) and compare with b). Estimate the $-3\,\text{dB}$ bandwidth of the AR(2) peak from r.

d) *Maximum entropy.* Given $r[0], \ldots, r[p]$, there are infinitely many PSDs consistent with these values. Burg proposed to choose the one with maximum entropy rate $h = \frac{1}{4\pi}\int_{-\pi}^{\pi} \ln P(\omega)\,\mathrm{d}\omega$ (+const.). Show with a Lagrange argument that this PSD has the form $P(\omega) = 1/\sum_{|m|\le p} \lambda_m e^{-j\omega m}$, i.e. it is an AR($p$) spectrum. How does the MEM spectrum treat the unknown lags $|m| > p$, compared with a (windowed) periodogram?

e) Show that Burg's reflection coefficients always satisfy $|k_m| \le 1$. Why does this guarantee a stable (minimum-phase) $A(z)$ and why is that important? Does the Yule–Walker method with the *biased* autocorrelation estimate have the same property?

Exercise 11.2 MEM vs. periodogram on short knock windows

JSLINUX

```
gcc -O2 -o memspec memspec.c -lm && ./memspec [N] [p]     # default N = 64, p = 8
```

a) Complete `ar_levinson()`, `ar_burg()` and `ar_psd()` in `ar.h`. Test them with the AR(2) example of 11.1 b) (you must get $a_1 = 0.1194$, $a_2 = 0.9048$).

b) Part (a) of `memspec` takes one knock window (N samples starting 10° after TDC, 3000 rpm, mean removed) and computes the Hann periodogram (zero padded to 1024), the Yule–Walker and the Burg spectrum. Plot `mem.csv` for $N = 32, 64, 128$ and describe the differences.

c) Part (b) estimates the knock frequency (maximum between 5.5 and 7.5 kHz) in 300 windows for $N = 32, 64, 128$ with four methods and reports mean, standard deviation, rms error, the $-3\,\text{dB}$ width of the peak and how often the second mode (10.5 kHz) is found. Interpret the table: in which respect is MEM better than the periodogram, in which not? Why does Burg's frequency estimate scatter more than the periodogram's?

d) Complete `ar_crit()`. Part (c) applies FPE, AIC and MDL to Burg models up to order 20. Which orders are chosen, and why are about 6–8 poles plausible for the knock signal?

e) Part (d) uses Burg with orders 2 to 32 for $N = 64$. Explain the growing number of peaks and the "split" knock peak, and give a rule of thumb for the order.

Exercise 11.3 Resolving two close resonances

JSLINUX

A temperature-dependent shift of the knock mode in one cylinder, or a second structural mode of the engine block, can create two resonances only a few hundred Hz apart. Can the ECU still separate them within one knock window?

```
gcc -O2 -o resolve resolve.c -lm && ./resolve 500 && ./resolve 250 && ./resolve 1000
```

a) Complete `resolved()`: the two components count as resolved if there is a local maximum within $\Delta f/2$ of f_1 and one within $\Delta f/2$ of f_2, and the spectrum dips by at least 3 dB between them.

b) The program generates two sinusoids at $f_1 = 6.5\,\mathrm{kHz}$ and $f_1 + \Delta f$ (random phases) in white noise with an SNR of 40, 20 and 10 dB per component and records how often the rectangular periodogram, the Hann periodogram and Burg ($p = N/3$, at most 32) resolve them for $N = 16 \ldots 256$. Determine the minimum record length for $\geq 90\,\%$ resolution for $\Delta f = 500\,\mathrm{Hz}$ and compare with the Rayleigh limit $N \approx f_s/\Delta f$. How does the SNR influence the result?

c) Repeat for $\Delta f = 250$ and $1000\,\mathrm{Hz}$. At $\Delta f = 500\,\mathrm{Hz}$ and 6000 rpm, a 60° window has 42 samples: which method can resolve the two modes?

d) Part (b) of the program uses *damped* modes with $\tau = 0.8\,\mathrm{ms}$, as in a real knock burst. Explain why now even long records and high SNR do not help for $\Delta f = 250\,\mathrm{Hz}$ and why Burg's success rate saturates at about 50–60 % for $\Delta f = 500\,\mathrm{Hz}$.

Deliverables

11.1 on paper. Completed `ar.h`; plots of `mem.csv` for $N = 32, 64, 128$; the tables of `memspec` parts (b)–(d) and of `resolve` for $\Delta f = 250, 500, 1000\,\mathrm{Hz}$ with a half-page interpretation: when would you use MEM in a knock controller, and when not?

Digital Signal Processing Exercise Sheet 12

Knock Detection

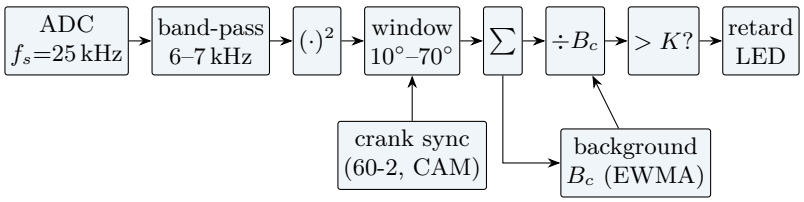
Script: Chapter 12 (capstone: angle-synchronous knock detection)

Learning goals

- Assemble the methods of the previous sheets into a complete, angle-synchronous knock detector and justify every block quantitatively.
- Evaluate a detector statistically on thousands of labelled combustions: ROC curve, area under the curve, detection rate at a fixed false-alarm rate, choice of the threshold.
- Compare energy-, FFT- and MEM-based knock features and quantify the contribution of band-pass, angle window and background normalisation.
- Run the detector in real time on the ESP32 with crank synchronisation, per-cylinder knock indication and ignition retard; optionally port it to the 8-bit Uno in fixed point.

The signal chain

The ECU does not see the engine. It sees only signals. The ECU never sees the knock – only a vibration signal that also contains noise, the combustion vibration and valve impacts. The standard knock detector of a production ECU is the chain below; every block is one of the previous sheets.



Conventions: $f_s = 25\,\text{kHz}$ (Sheet 10), band-pass = the 10th-order Butterworth biquad cascade of `knock_filter.h` (Sheet 10; group delay 22 samples at 6.5 kHz) or the single biquad `kf_bp2` ($B = 400\,\text{Hz}$, 20 samples), window 10°–70° after the TDC of the firing cylinder, feature $E_c = \sum_{\text{window}} y^2[n]$ for cylinder c, normalised $R_c = E_c/B_c$, knock if $R_c > K$. Ground truth: `e.knock_flag[c]` in the C model, the KNK pin of the engine-sim chip in Wokwi. Code: `code/jslinux/sheet12/` (`knockeval.c` plus `knock_filter.h`, `ar.h`, `fft.h` – use your own versions from Sheets 8, 10, 11 if you like) and `code/wokwi/sheet12_knock_esp32/`, `code/wokwi/sheet12_knock_uno/`.

Exercise 12.1 Detector design

PAPER

a) Justify every block of the chain with the respective chapter: why sampling at 25 kHz (and what about anti-aliasing), why an angle window and why exactly 10°–70° ATDC, why a band-pass, why squaring (instead of a linear operation), why summation, why a normalisation and why a threshold. Which blocks are LTI, which are not?

b) Compute the window length in samples at 1500, 3000 and 6000 rpm. By how many degrees must the window be shifted to compensate the group delay of the band-pass (22 samples)? Why is a fixed sample shift correct although the window is defined in degrees?

c) Energy budget. Show that a knock burst $A\mathrm{e}^{-t/\tau}\sin(2\pi f_1 t)$ that lies completely in the window and is passed with gain 1 contributes $E_k \approx f_s A^2 \tau/4$ (in V^2, sum over samples). White noise with standard deviation σ contributes on average $E_n = N_w \sigma^2 \cdot 2B_n/f_s$, where N_w is the window length and B_n the noise bandwidth of the band-pass ($B_n = \int_0^{f_s/2} |H(f)|^2\,\mathrm{d}f = 1161\,\text{Hz}$ for the cascade). Evaluate E_k for $A = 0.2\,\text{V}$ and E_n for $\sigma = 0.05$ and $0.15\,\text{V}$ at the three speeds. What do you expect for the detection performance?

d) Without knock, E_c is approximately $E_n \cdot \chi^2_\nu/\nu$ distributed with $\nu \approx 2B_nT_w$ degrees of freedom (T_w = window duration). Compute ν for the three speeds and, using the 99 % quantiles $\chi^2_{\nu,0.99}/\nu \approx 2.0$ ($\nu = 15.5$), 2.6 ($\nu = 7.7$), 3.4 ($\nu = 3.9$), the threshold for 1 % false alarms in units of the mean noise energy. Why does a fixed threshold K give an rpm-dependent false-alarm rate?

e) Background normalisation: propose an update rule for B_c that follows slow changes (sensor ageing, rpm, load) but is not pulled up by knocking combustions. Why one background per cylinder?

f) Control action: on a detected knock the ignition of that cylinder is retarded by Δ_r, without knock it is advanced again by $\Delta_a \ll \Delta_r$ per combustion. Explain the asymmetry and what a false alarm costs.

Exercise 12.2 Offline evaluation with ground truth JSLINUX

`knockeval.c` runs the virtual engine at 1500, 3000 and 6000 rpm with sensor noise $\sigma = 0.05$ and 0.15 V (6 scenarios, 1000 evaluated combustions each, knock probability 0.2, knock amplitude $0.2\,\mathrm{V}\times(0.5\ldots1.5)$, light knock) and computes for every combustion eight features, each normalised by its own per-cylinder background:

WB-seg / WB-win	wide-band energy $\sum x^2$ over the whole 180° segment / in the window
BP-seg / BP-win	band-pass (cascade) energy over the segment / in the delay-compensated window
BP2-win	as BP-win with the single biquad ($B = 400$ Hz)
FFT / FFT-rect	power 6–7 kHz of the zero-padded FFT of the raw window samples, Hann / no window
MEM	power 6–7 kHz of the Burg AR(8) spectrum of the raw window samples

```
gcc -O2 -o knockeval knockeval.c -lm && ./knockeval [knock_amp] [ncomb]
```

a) Complete the window logic: integrate x^2 in the window 10°–70° ATDC and y^2 (cascade, single biquad) while the *delayed* angle is in the window.

b) Complete the background update (clipped EWMA, faster during the first 16 combustions of each cylinder).

c) Complete the ROC computation in `roc()`: the combustions are sorted by decreasing feature value; every threshold between two values gives a point (FPR, TPR). Compute the AUC (trapezoidal rule) and the detection rate at ≤ 1 % false alarms.

d) Complete the FFT and MEM features (`fft.h`, `ar.h`).

e) Run the program and interpret the AUC and detection tables: what is the contribution of the band-pass, of the angle window, of the delay compensation, of the normalisation (compare the pooled ROC with and without)? Plot `roc.csv`.

f) Choose a threshold K for 1 % false alarms over all scenarios and compare the per-scenario thresholds with your prediction from 12.1 d).

g) Compare the energy, FFT and MEM features. Why is the Hann-windowed FFT so much worse at 1500 rpm than the rectangular one? Which feature would you implement on the microcontroller, and why?

Exercise 12.3 Real-time knock detector WOKWI

Open `code/wokwi/sheet12_knock_esp32/` (ESP32 + engine-sim chip, 4 knock LEDs, retard LED, logic analyzer). Pins: KNOCK → GPIO34, CRANK → 18, CAM → 19, TDC → 21 (only to check the synchronisation), KNK → 22 (ground truth, only for the evaluation); LEDs cylinder 1–4 on 25,

26, 27, 14; retard PWM on 13. The sketch samples at 25 kHz (deadline loop), filters with `kf_sos` and prints one line `cyl,rpm,R,knock,truth,retard` per combustion and every 200 combustions the confusion counts.

a) *Crank sync.* Complete the 60-2 decoder in the CRANK interrupt (gap detection: a tooth period more than twice the previous one; revolution from the CAM level at tooth 10) and the angle interpolation `crank_angle()`. The sketch compares the decoded angle at every TDC pulse with 120/300/480/660° and prints the mean error. What limits the accuracy?

b) *Window.* Complete the integration of y^2 while the delayed angle (ring buffer `th_hist`, `KF_DELAY` = 22 samples) is in the window.

c) *Decision and action.* Complete `finish_combustion()`: $R = E/B_c$, knock if $R > K$ ($K = 2$ from 12.2), background update, ignition retard (+1.5° per knock, −0.05° per knock-free combustion, max. 9°).

d) Run the detector at 1500, 3000, 6000 rpm with knock amplitude 1.0 and 0.2 V and noise 50 and 150 mV (knock probability 20 %). Report detection and false-alarm rates and compare them with 12.2. Watch the LEDs and the retard output.

e) Estimate the processing time per sample on the ESP32 and the UART load at 6000 rpm. Why must the sketch not print the samples?

f) The engine-sim chip has no spark input: the retard does not influence the knock probability. Describe how a closed knock control loop would behave and how you would test it.

Exercise 12.4 Port to the Arduino Uno

CHALLENGE

Port the detector to the Uno (`code/wokwi/sheet12_knock_uno/`, starting from the Q15 biquad of Sheet 10): Timer1 ISR at 25 kHz, single biquad `kf_bp2_q14`, window from the TDC pulse (period in samples of the previous combustion, cylinder 1 = TDC while CAM is HIGH), energy $\sum(y^2 \gg 8)$ in 32 bit, integer background $B \mathrel{+}= (\min(E, 3B) - B) \gg 4$, decision $E > 2B$ without division. Report detection and false-alarm rates as in 12.3 and the limits of the port.

Deliverables

12.1 on paper (block justification, numbers for b)–d)). `knockeval`: AUC and detection tables, pooled ROC plot (BP-win, WB-win, FFT, MEM), chosen threshold with justification. Wokwi: the confusion counts for the settings of 12.3 d) and a screenshot of the LEDs; a short discussion (max. one page) of the remaining weaknesses of your detector.